2026-2027 9. Sınıf Programlamanın Temelleri

İçindekiler

I Öğrenme Birimi Algoritmik Düşünme ve Tasarım 1

1. Problem Analizi ve Bileşenleri 1
   - 1.1. Algoritma ve Tasarım Kriterleri 1
   - 1.2. Problemi Bileşenlerine Ayırma 1
   - 1.3. Girdi-İşlem-Çıktı Modeli 1
2. Akış Şeması (Flowchart) 2
   - 2.1. Sözde Kod ve Elle Çalıştırma 2
     - 2.1.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1) 2
     - 2.1.2. Karşılaştırma Operatörleri ve Kullanım Örnekleri 3
       - 2.1.2.1. Karşılaştırma Operatörleri İle Yapılan İşlem Örnekleri 3
     - 2.1.3. Örnek 1 Hatalı Giriş Kontrolü Algoritması (Sayfa 28 Örnek 4) 4
     - 2.1.4. Örnek 2 Sıcaklık Kontrolü (Sayfa 28 Örnek 3) 5
     - 2.1.5. Örnek 3: Yaş Kontrolü ve Ehliyet Alabilme (Sayfa 25 Örnek 2) 6
     - 2.1.6. Flowgorithm ve Thonny Uygulamaları 7
     - 2.1.7. Değişkenler ve Veri Tipleri 8
       - 2.1.7.1. Temel Veri Tipleri 8
       - 2.1.7.2. Değişken İsimlendirme Kuralları 8
   - 2.2. Akış Şeması (Flowchart) Sembolleri ve Görsel Modelleme 9
     - 2.2.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1) 9
     - 2.2.2. Örnek 2: İki Sayının Ortalamasını Bulma 9
     - 2.2.3. Örnek Çalışma Soruları 10
     - 2.2.4. Karar Yapıları 10
     - 2.2.5. Örnek 3 Sıcaklık Kontrolü 11
     - 2.2.6. Örnek 4 Hatalı Giriş Kontrolü Algoritması 12
     - 2.2.7. Örnek 5: Yaş Kontrolü ve Ehliyet Alabilme 13
     - 2.2.8. Örnek Çalışma Soruları Karar Yapıları 14
     - 2.2.9. Döngü(Loop) 15
     - 2.2.10. Örnek 6 Döngüsel Ekran Yazımı (Sayfa 37 Örnek 8) 15
     - 2.2.11. Örnek 7 Ardışık Sayıların Toplamı (Sayfa 38 Uygulama 4) 16
     - 2.2.12. Örnek Çalışma Soruları Döngü Konusu 18
   - 2.3. Algoritmalarda Metinsel Veriler ve İşlemler 19
     - 2.3.1. Örnek Metin Birleştirme (Sayfa 42 Örnek 9) 19
   - 2.4. Dijital Depo 19

II Temel Programlama Yapıları 21

1. Donanımın Temeli: Transistör ve İkili Sistem (Binary) 21
2. İlk Bilgisayarlar ve 8 Bitlik Komut Girişleri (Altair 8800) 21
3. Onaltılık Sayı Sistemi (Hexadecimal) 22
4. Metin Temsili: 8 Bit = 1 Byte ve ASCII Tablosu 22
5. Bit'ten Terabayt'a Veri Büyüklükleri 23
6. Makine Kodundan Assembly Diline 24
7. Programlama Dilleri 25
8. Değişkenler ve Bellek(RAM) 27
9. Veri Türleri ve RAM Kullanımı 30
   - 9.1. Boolean (bool): doğru / yanlış 30
   - 9.2. Karakter (char) 30
   - 9.3. Tam Sayı (int) 31
   - 9.4. Ondalıklı Sayı (float) 31

10. Bellek Yönetimi, RAM ve İşlemcinin İlişkisi 32
11. Karakterden Metne (Karakter Dizisi) 33
12. Python Dili 37
13. Tür Dönüşümü (Type Conversion / Type Casting) 37
13.1. Kullanıcı Girişlerinin Sayıya Dönüştürülmesi (Metin → Sayı) 37
13.2. Otomatik (Örtük / Implicit) Dönüşüm: 38
13.3. Bilinçli (Açık / Explicit) Dönüşüm: 38
13.3.1. Hassasiyeti Korumak İçin Tam Sayıyı Kesrli Sayıya Çevirmek (int ⟶ float): 38
13.3.2. Ondalık Kısmı Atmak İçin (float ⟶ int): 39
14. Değişken Tanımlama Kuralları 39
15. Değişken Adlandırma Standartları (Notasyonlar) 40
15.1. Yılan Gösterimi (snake_case) 40
15.2. Camel Case (Deve Notasyonu) Kuralı 40
16. Sabitler 40
17. Açıklama Satırı (Comment line) 40
18. Operatörler ve İşlem Önceliği 41
18.1. Atama Operatörleri 42
18.1.1. Atama Operatörü (**=**): Değişken Oluşur veya Güncellenir 42
18.1.2. Bileşik Atama Operatörleri 43
18.2. Aritmetik Operatörler 43
18.3. Aritmetik Operatörlerde İşlem Önceliği 45
18.4. Karşılaştırma Operatörleri 46
18.5. Mantıksal Operatörler 47
18.6. Operatörler Arası İşlem Önceliği 48
19. Fonksiyonların Kullanımı 49
19.1. Yerleşik (Built-in) Fonksiyonlar 50
19.1.1. Dönüş Değeri Olan ve Olmayan Fonksiyonlar 51
20. input() ve print() Fonksiyonları 51
20.1. print(): Ekrana Yazdırma 51
20.2. input(): Kullanıcıdan Veri Alma 52
20.2.1. input() Her Zaman Metin Verir 52
20.3. print() ile Metin ve Değişkenleri Birlikte Yazdırma 53
20.3.1. Virgül (,) ile Yazdırma 53
20.3.2. f-string (Formatlı Metin) 53
20.3.3. f-string İçinde İşlem Yapmak 54
20.3.4. f-string ile Sayıları Biçimlendirme 54
20.3.5. f-string ile Hizalama 54
20.3.6. f-string'de Süslü Parantez ve Tırnak Kullanımı 55
20.3.7. Hata Ayıklarken Kullanışlı: {degisken=} 55
20.4. print() Fonksiyonunun sep ve end Parametreleri 55
20.4.1. sep: Değerler Arasına Ne Gelsin? 55
20.4.2. end: Satır Nasıl Bitsin? 56
20.5. Kaçış Karakterleri (Escape Characters) 56
20.6. Uygulama Örnekleri 56
III Python `type()` Fonksiyonu 58
1. `type()` Nedir? 58
2. Temel Veri Türleri 59
3. Neden İhtiyaç Duyarız? 59
3.1. Tipik hata: `input()` her zaman `str` döndürür 59
4. Tür Kontrolü: `type()` ve `isinstance()` 60
4.1. Birden fazla tür kontrol etmek 60

5. Türü Kullanarak Nesne Oluşturmak 60
6. İleri Düzey: Üç Argümanlı `type()` 61
7. Alıştırmalar 62
8. `int(input())` Nasıl Çalışır? 63
    8.1. Adım 1: `input()` çalışır ve `int()` fonksiyonuna gider 63
    8.2. Adım 2: `int()` çalışır ve sonuç `puan` değişkenine gider 63
    8.3. Sonuç 63

2026-2027 9. Sınıf Programlamanın Temelleri

I Öğrenme Birimi Algoritmik Düşünme ve Tasarım

1. Problem Analizi ve Bileşenleri

ANALİZ

Bir olayı veya durumu incelemek ve ayrıntılı biçimde anlamaya çalışmaktır.

Programlama sürecinde problem analizi, çözülmesi gereken sorunu dikkatlice inceleme aşamasıdır. Bu aşamada sorunun ne olduğu, çözüm için hangi bilgilere ihtiyaç duyulduğu, bu bilgilerle neler yapılacağı ve sonunda hangi sonucun elde edilmek istendiği belirlenir. Örneğin okul kantinindeki sıra sorununda öğrenci sayısı ve alışveriş süreleri incelenir. Daha sonra bekleme süresini azaltacak bir sıra düzeni planlanır.

1.1. Algoritma ve Tasarım Kriterleri

Algoritma, belirli bir problemi çözmek veya hedeflenen bir amaca ulaşmak için tasarlanan, açık ve sıralı adımlardır. Algoritmadaki adımlar açık olmalı, sırayla uygulanmalı ve bir sonuca ulaşmalıdır.

Örnek: İki sayının toplamını bulma;

1. Birinci sayıyı al.
2. İkinci sayıyı al.
3. İki sayıyı topla.
4. Sonucu ekrana yazdır.

Örneğin girilen sayılar 8 ve 5 ise, ekranda 13 sonucu gösterilir.

1.2. Problemi Bileşenlerine Ayırma

Problemi Bileşenlerine Ayırma, büyük ve karmaşık bir problemi daha küçük ve yönetilebilir parçalara bölme yöntemidir. Her parça ayrı ayrı incelenir ve çözülür. Bu yöntem, hataları bulmayı, sistemi geliştirmeyi ve güncellemeyi kolaylaştırır.

Örnek: Dijital oyun tasarlama : Oyunun karakter hareketleri, sesleri ve puan sistemi ayrı bölümler olarak hazırlanır. Bu bölümler daha sonra birleştirilir. Böylece oyunu geliştirmek ve hataları bulmak kolaylaşır.

1.3. Girdi-İşlem-Çıktı Modeli

Bilgisayarda problemler genellikle Girdi–İşlem–Çıktı modeliyle çözülür.

Girdi: Sisteme verilen bilgiler veya verilerdir.

İşlem: Bu bilgiler üzerinde yapılan hesaplama ve kontrollerdir.

Çıktı: İşlem sonunda elde edilen sonuçtur.

Örnek: Bankamatikten para çekme Kullanıcının kartı, şifresi ve çekmek istediği para miktarı girdidir. Bankamatik şifreyi kontrol eder ve yeterli bakiye olup olmadığına bakar; bu bölüm işlemdir. Para, makbuz ve ekranda gösterilen yeni bakiye ise çıktıdır.

2. Akış Şeması (Flowchart)

Akış şeması, bir algoritmanın kod yazılmadan önce semboller ve çizimler kullanılarak gösterilmesidir. Çözüm adımlarını daha kolay anlamayı, kontrol etmeyi ve hataları kodlamaya başlamadan fark etmeyi sağlar.

2.1. Sözde Kod ve Elle Çalıştırma

Sözde kod, bir algoritmayı herhangi bir programlama dilinin kurallarına bağlı kalmadan, günlük dile yakın ve açık komutlarla yazma yöntemidir. Sözde kodlar, programcının mantık hatalarına odaklanmasını kolaylaştırır.

Dry run(Elle Çalıştırma), bir algoritmayı kodlamadan önce kağıt üzerinde adım adım çalıştırma yöntemidir. Bu sırada değişkenlerin değerleri takip edilir ve sonucun doğru olup olmadığı kontrol edilir. Böylece mantık hataları erken fark edilir.

2.1.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1)

Kullanıcı tarafından girilen iki sayının toplamını ekrana yazdıran sözde kodu ve elle çalıştırma tablosunu hazırlayın. Birinci ve ikinci sayı için kullanıcıdan alınan değerler için sırasıyla **s1** ve **s2** değişkenlerini, toplam için **toplam** değişkenini kullanın.

Sözde Kod

1. Adım: BAŞLA.
2. Adım: Birinci sayıyı (**s1**) kullanıcıdan al.
3. Adım: İkinci sayıyı (**s2**) kullanıcıdan al.
4. Adım: **toplam = s1 + s2** işlemini yap.
5. Adım: toplam **sonucunu ekrana yazdır**.
6. Adım: DUR.

Elle Çalıştırma Tablosu

Örnek Girdiler: s1 = 15, s2 = 25 **Beklenen Çıktı:** toplam = 40

Adım	İşlem	s1	s2	toplam	Açıklama
1	BAŞLA	-	-	-	Algoritma başlar
2	**s1** değerini oku.	**15**	-	-	Kullanıcı 15 sayısını girer
3	**s2** değerini oku.	15	**25**	-	Kullanıcı 25 sayısını girer
4	**toplam = s1 + s2**	15	25	**40**	15 + 25 = 40 işlemi gerçekleşir
5	toplam değerini yazdır	15	25	40	Ekrana 40 sonucu yazılır
6	DUR	15	25	40	Algoritma başarıyla sonlanır

2.1.2. Karşılaştırma Operatörleri ve Kullanım Örnekleri

Algoritmalar; günlük hayatta olduğu gibi belirli koşullara göre farklı yollara sapabilir. Yazılımcılar, iki veriyi birbiriyle kıyaslayan ve aralarındaki mantıksal ilişkiyi belirleyen **karşılaştırma operatörlerini** kullanarak **Doğru**(True) veya **Yanlış**(False) mantıksal sonucunu hesaplar.

Aşağıda ki tabloda sık karşılaştırma operatörleri ve kullanımı gösterilmektedir.

Operatör	Anlamı	Örnek Kullanım	Bilgisayar Dilindeki Karşılığı ve Açıklama
==	Eşittir	tahmin == 1234	**tahmin** değeri, **1234** sayısına eşitse sonuç **Doğru** olur.
!=	Eşit Değildir	sifre != “123”	Şifre, “123” değerine eşit değilse sonuç **Doğru** olur.
>	Büyüktür	yas > 18	Yaş değeri, 18’den büyükse sonuç **Doğru** olur.
<	Küçüktür	sicaklik < 0	Sıcaklık, 0’dan küçükse sonuç **Doğru** olur.
>=	Büyük Eşittir	puan >= 50	Puan, 50 veya 50’den daha büyükse sonuç **Doğru** olur.
<=	Küçük Eşittir	hiz <= 90	Hız, 90 veya 90’dan daha küçükse sonuç **Doğru** olur.

2.1.2.1. Karşılaştırma Operatörleri İle Yapılan İşlem Örnekleri

EŞİTTİR (==)

- **5 == 5** → Sonuç: true (1)
- **12 == 20** → Sonuç: false (0)
- **“Tuna” == “Tuna”** → Sonuç: true (1)
- **“Ahmet” == “ahmet”** → Sonuç: false (0) (Büyük/küçük harf duyarlılığı)
- **–10 == 10** → Sonuç: false (0)

EŞİT DEĞİLDİR (!=)

- **10 != 5** → Sonuç: true (1)
- **7 != 7** → Sonuç: false (0)
- **“Java” != “Python”** → Sonuç: true (1)
- **“Okul” != “Okul”** → Sonuç: false (0)
- **–4 != 4** → Sonuç: true (1)

BÜYÜKTÜR (>)

- **15 > 8** → Sonuç: true (1)
- **3 > 10** → Sonuç: false (0)
- **5 > 5** → Sonuç: false (0)
- **–2 > –10** → Sonuç: true (1)
- **0.5 > 0.2** → Sonuç: true (1)

KÜÇÜKTÜR (<)

- **4 < 9** → Sonuç: true (1)
- **25 < 12** → Sonuç: false (0)
- **7 < 7** → Sonuç: false (0)
- **–15 < –3** → Sonuç: true (1)
- **1.25 < 1.30** → Sonuç: true (1)

BÜYÜK EŞİTTİR (>=)

- **18 >= 18** → Sonuç: true (1)
- **20 >= 15** → Sonuç: true (1)
- **6 >= 10** → Sonuç: false (0)
- **0 >= 0** → Sonuç: true (1)
- **–5 >= –5** → Sonuç: true (1)

KÜÇÜK EŞİTTİR (<=)

- **5 <= 5** → Sonuç: true (1)
- **3 <= 8** → Sonuç: true (1)
- **14 <= 9** → Sonuç: false (0)
- **–8 <= –2** → Sonuç: true (1)
- **100 <= 99** → Sonuç: false (0)

2.1.3. Örnek 1 Hatalı Giriş Kontrolü Algoritması (Sayfa 28 Örnek 4)

Kullanıcının önceden belirlediği bir şifre ile sisteme girişini kontrol eden programının sözde kodunu ve elle çalıştırma tablosunu hazırlayın. Sistem şifresi için **sistemKey**, kullanıcı şifresi için **kullaniciKey** kullanılacak.

Sözde Kod

1. Adım: BAŞLA
2. Adım: **sistemKey = 1234**
3. Adım: kullanıcıdan (**kullaniciKey**) değerini al
4. Adım: **EĞER sistemKey == kullaniciKey**
   EVET ise "Sisteme Girer"
   HAYIR ise "Sisteme Giremez!"
5. Adım: BİTİR

Elle Çalışma Tablosu

Örnek Girdiler: sistemKey=1234, kullaniciKey=9876, **Beklenen Çıktı:** Sisteme Giremez

Ad	İşlem	sistemKey	kullaniciKey	Koşul sistemKey == kullaniciKey	Çıktı	Açıklama
1	BAŞLA	-	-	-	-	Algoritma başlar.
2	sistemKey = 1234	**1234**	-	-	-	Kodun içinde sistem şifresi **sistemKey=1234** oluşturuldu
3	kullaniciKey = 9876 Gir	1234	**9876**	-	-	Kullanıcıdan **kullaniciKey=9876** değeri alındı.
4	sistemKey == kullaniciKey	1234	9876	**HAYIR**	**Giriş yapılamaz!**	Sistemde bulunan şifre ile kullanıcının girdiği şifre karşılaştırılır. Kullanıcı sisteme giremez!
5	DUR	1234	9876	-	-	Algoritma başarıyla sonlanır.

Elle Çalışma Tablosu

Örnek Girdiler: sistemKey=1234, kullaniciKey=1234, **Beklenen Çıktı:** Sisteme Girer

Ad	İşlem	sistemKey	kullaniciKey	Koşul sistemKey == kullaniciKey	Çıktı	Açıklama
1	BAŞLA	-	-	-	-	Algoritma başlar.
2	sistemKey = 1234	**1234**	-	-	-	Kodun içinde sistem şifresi **sistemKey=1234** oluşturuldu
3	kullaniciKey = 1234 Gir	1234	**1234**	-	-	Kullanıcıdan **kullaniciKey=1234** değeri alındı.
4	sistemKey == kullaniciKey	1234	1234	**EVET**	**Giriş yapılır.**	Sistemde bulunan şifre ile kullanıcının girdiği şifre karşılaştırılır. Kullanıcı sisteme girer.
5	DUR	1234	1234	-	-	Algoritma başarıyla sonlanır.

2.1.4. Örnek 2 Sıcaklık Kontrolü (Sayfa 28 Örnek 3)

Ortam sıcaklığı (**SICAKLIK**) 20 derecenin altına düştüğünde kombiyi çalıştıran, 20 derecenin üzerine çıktığında ise kombiyi durduran bir termostat programının sözde kodunu ve elle çalıştırma tablosunu hazırlayın.

Sözde Kod

1. Adım: BAŞLA
2. Adım: Sıcaklık değeri oku **SICAKLIK**
3. Adım: **EĞER SICAKLIK < 20**
   EVET ise "Kombi Çalıştır"
   HAYIR ise "Kombi Durdu"
4. Adım: BİTİR

Elle Çalışma Tablosu

Örnek Girdiler: SICAKLIK = 18, **Beklenen Çıktı:** Kombi Çalışır

Adım	İşlem	SICAKLIK	Koşul (SICAKLIK < 20)	Çıktı	Açıklama
1	BAŞLA	-	-	-	Algoritma başlar.
2	SICAKLIK oku	**18**	-	-	Ortamdaki sıcaklık değeri okunur, SICAKLIK değerine atanır
3	SICAKLIK < 20	18	**EVET**	**Kombi Çalışır!**	Koşul Evet
4	DUR	18	-	-	Algoritma başarıyla sonlanır.

Örnek Girdiler: SICAKLIK = 22, **Beklenen Çıktı:** Kombi Durur

Adım	İşlem	SICAKLIK	Koşul (SICAKLIK < 20)	Çıktı	Açıklama
1	BAŞLA	-	-	-	Algoritma başlar.
2	SICAKLIK oku	**20**	-	-	Ortamdaki sıcaklık değeri okunur, SICAKLIK değerine atanır
3	SICAKLIK < 20	20	**HAYIR**	**Kombi Durur!**	Koşul Hayır
4	DUR	18	-	-	Algoritma başarıyla sonlanır.

2.1.5. Örnek 3: Yaş Kontrolü ve Ehliyet Alabilme (Sayfa 25 Örnek 2)

Kullanıcıdan o günün yılını (**sistemYil**) ve kullanıcıdan doğum yılını (**dogumYil**) alarak kullanıcının 18 yaşından büyük olması durumunda "**Ehliyet Alır!**" küçük olması durumunda "**Ehliyet Almaz!**" yazısını ekrana yazdıran programın sözde kodunu ve elle çalışma tablosunu hazırlayınız.

Sözde Kod

1. Adım: BAŞLA.
2. Adım: Sistem Tarihi (**sistemYil**) değerini oku.
3. Adım: Kullanıcının Doğum Yılı (**dogumYil**) değerini oku.
4. Adım: **yas = sistemYil - dogumYil** işlemini yap.
5. Adım: **Eğer yas >= 18**
   EVET ise "**Ehliyet Alır!**" mesajını yazdır.
   HAYIR ise "**Ehliyet Almaz!**" mesajını yazdır.
6. Adım: DUR.

Elle Çalışma Tablosu

Örnek Girdiler: Sistem Tarihi (sistemYil) = 2026, Doğum Yılı (dogumYil) = 2005, **Beklenen Çıktı:** Ehliyet alabilirsiniz.

Adım	İşlem	sistemYil	dogumYil	yas	Koşul (yas >= 18)	Çıktı	Açıklama
1	BAŞLA	-	-	-	-	-	Algoritma başlar.
2	sistemYil oku	**2026**	-	-	-	-	Sistem tarihi alınır.
3	dogumYil oku	2026	**2005**	-	-	-	Kullanıcı 2005 sayısını girer.
4	yas =sistemYil - dogumYil	2026	2005	**21**	-	-	2026 - 2005 = 21
5	yas >= 18	2026	2005	21	**Evet**	**Ehliyet Alır!**	Koşul doğru → mesaj yazdırılır.
6	DUR	2026	2005	21	-	-	Algoritma başarıyla sonlanır.

Örnek Girdiler: Sistem Tarihi (sistemYil) = 2026, Doğum Yılı (dogumYil) = 2010, **Beklenen Çıktı:** Ehliyet almak için henüz küçüksünüz.

Adım	İşlem	sistemYil	dogumYil	yas	Koşul (yas >= 18)	Çıktı	Açıklama
1	BAŞLA	-	-	-	-	-	Algoritma başlar.
2	sistemYil oku	**2026**	-	-	-	-	Sistem tarihi alınır.
3	dogumYil oku	2026	**2010**	-	-	-	Kullanıcı 2010 sayısını girer.
4	yas =sistemYil - dogumYil	2026	2010	**16**	-	-	2026 - 2010 = 16
5	yas >= 18	2026	2010	16	**Hayır**	**Ehliyet Almaz!**	Koşul yanlış → mesaj yazdırılır.
6	DUR	2026	2010	16	-	-	Algoritma başarıyla sonlanır.

2.1.6. Flowgorithm ve Thonny Uygulamaları

Derslerimizde kullanacağımız yazılımların kurulum bağlantılarına ve destekleyici eğitim videolarına yan taraftaki karekodları taratarak veya altlarındaki bağlantılara tıklayarak doğrudan erişebilirsiniz.

Flowgorithm Programı

Algoritmik düşünme becerimizi geliştirmek ve tasarladığımız mantıksal adımları görselleştirmek için Flowgorithm yazılımını kullanacağız. Bu program sayesinde yazdığımız akış şemalarını adım adım çalıştırabilir, değişkenlerin değer değişimlerini anlık olarak izleyebilir ve algoritmalarımızı kod yazmadan önce test edebiliriz.

Flowgorithm Programı

Bağlantıya tıklayın
flowgorithm.org

Flowgorithm Youtube Eğitimi

Derste işlediğimiz konuları pekiştirmek ve kendi hızınızda pratik yapabilmeniz için Atilla Çetin tarafından hazırlanan Flowgorithm ile Algoritma ve Akış Diyagramları video serisinden yararlanabilirsiniz. Temel kavramlardan uygulamalı örneklere kadar uzanan bu oynatma listesine sağdaki karekodu okutarak veya bağlantıya tıklayarak ulaşabilirsiniz.

Youtube Eğitimi

Bağlantıya tıklayın
kisa.link/bHGEg

Thonny IDE

Akış şemalarından kodlamaya geçerken, Python öğrenimi için özel olarak tasarlanmış Thonny IDE'yi kullanacağız. Sade arayüzü ve adım adım kod takibi sunan hata ayıklama (debug) özellikleriyle başlangıç seviyesine son derece uygundur. Dahili Python desteği sayesinde harici bir kurulum gerektirmez; yalnızca Thonny'yi kurarak kodlarınızı hemen çalıştırmaya başlayabilirsiniz.

Thonny

Bağlantıya tıklayın
thonny.org

Programlamanın Temelleri Ders Notları

Derslerimizde işlediğimiz konuları düzenli takip edebilmeniz ve tekrar yapabilmeniz için hazırladığım bu doküman; MEB'in 14 Eylül 2026 tarihli **'Programlamanın Temelleri 9'** müfredatını temel almaktadır. Konu anlatımları, akış şemaları ve örnek kod uygulamalarıyla ders içi kazanımları pekiştirmeyi amaçlayan kapsamlı bir çalışma kaynağıdır

Ders Notları

Bağlantıya tıklayın
kisa.link/hpYCg

2.1.7. Değişkenler ve Veri Tipleri

DEĞİŞKEN NEDİR?

Programın çalışması sırasında kullanıcıdan alınan, hesaplanan veya geçici olarak hafızada (RAM) tutulması gereken verileri saklayan isimlendirilmiş bellek alanlarıdır.

Flowgorithm üzerinde bir değişkeni kullanmadan önce mutlaka **Tanımlama (Declare)** adımıyla bilgisayara tanıtmak gerekir. Aynı türdeki birden fazla değişken aralarına virgül konularak tek seferde tanımlanabilir (Örn: `sistemYil, dogumYil, yas`).

2.1.7.1. Temel Veri Tipleri

Flowgorithm içerisinde en sık kullanılan 4 temel veri tipi bulunur:

Veri Tipi	Türü	Örnek Değerler	Açıklama
Tamsayı (Integer)	Tam Sayı	`15`, `0`, `-8`, `2026`	Küsüratsız, negatif veya pozitif tam sayı değerlerini tutar. Yaş, adet, sayaç gibi veriler için kullanılır.
Reel (Real / Float)	Ondalıklı Sayı	`3.14`, `18.5`, `-0.75`	Kesirli veya ondalıklı sayıları tutar. Ortalama, boy, sıcaklık veya fiyat gibi hassas hesaplamalarda tercih edilir.
String	Metinsel Veri	`"Ahmet"`, `"1234"`, `"A"`	Çift tırnak içinde yazılan her türlü yazı, harf, sembol ve rakamı saklar. **Matematiksel işlemlere girmez.**
Boolean	Mantıksal	`true(1)`, `false(0)`	Yalnızca iki değerden birini alabilir: **Doğru (true(1))** ya da **Yanlış (false(0))**. Karar ve döngü kontrollerinde kullanılır.

2.1.7.2. Değişken İsimlendirme Kuralları

Değişken tanımlarken programların hata vermemesi ve kodun düzenli olması için belirli kurallara uymak şarttır.

Türkçe Karakter Kullanımı

Kodlama dillerinin ve yazılım araçlarının küresel standardı gereği, değişken isimlendirirken
Türkçe'ye özgü harfleri kullanmaktan kesinlikle kaçınınız!.
Aşağıdaki tabloda büyük ve küçük Türkçe karakterlerin yerine İngilizce alfabe karşılıkları tercih edilmelidir:

Türkçe Karakter	Doğru Kullanım	İngilizce Karakte
ç / Ç	→	**c / C**
ğ / Ğ	→	**g / G**
ı / İ	→	**i / I**
ö / Ö	→	**o / O**
ş / Ş	→	**s / S**
ü / Ü	→	**u / U**

Camel Case (Deve Notasyonu) Kuralı

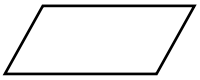
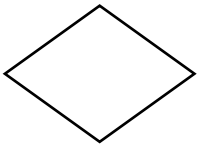
Değişken adları tanımlanırken kelimeler arasında boşluk bırakılamaz. Birden fazla kelimeden oluşan değişkenleri okunaklı kılmak için dünya çapında standart olan **camelCase** yöntemi kullanılır.
Kural: İlk kelimenin tüm harfleri küçük başlar. Sonraki her kelimenin ilk harfi büyük yazılır (kelimelerin ortasındaki hörgüç görünümü gibi).

Yanlış Biçim	camelCase	Açıklama
`sistem yil`	`sistemYil`	Boşluk yerine birleşik yazılır
`dogumyil`	`dogumYil`	İkinci kelimenin baş harfi büyük
`ogrencinotortalamasi`	`ogrenciNotOrtalamasi`	Okunabilirlik artırılır

2.2. Akış Şeması (Flowchart) Sembolleri ve Görsel Modelleme

Akış şeması(flowchat), bir algoritmanın adımlarını semboller ve oklarla gösteren görsel bir modeldir. Programın çalışma mantığını kolayca anlamayı ve kodlama öncesinde olası hataları fark etmeyi sağlar.

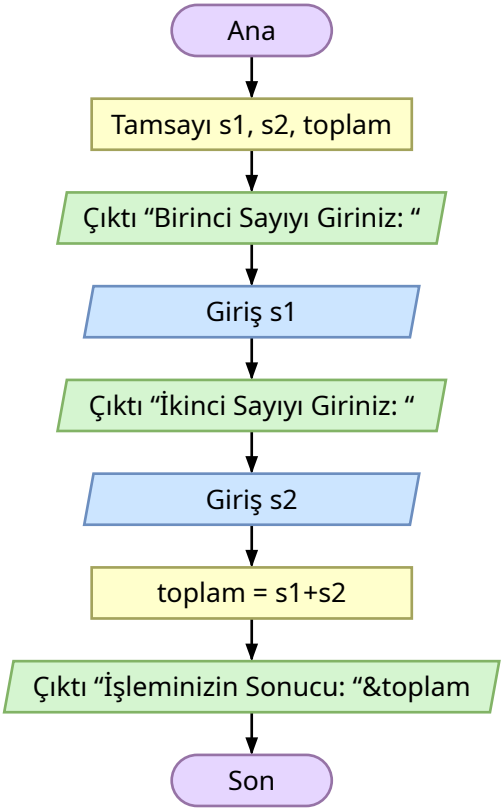
Tablo 1.2: Temel Akış Şeması Sembolleri, İşlevleri ve Günlük Hayat Karşılıkları.

Sembol	Adı	İşlevi	Günlük Hayat Karşılığı
	Oval (Terminal)	Algoritmanın başlangıcını ve bitişini temsil eder.	Bir işe başlama veya sonlandırma anıdır.
	Paralelkenar (Giriş / Çıkış)	Kullanıcıdan veri alınacağını veya ekrana bilgi yazdırılacağını belirtir.	Soru sormak veya cevap vermek içindir.
	Dikdörtgen (İşlem)	Matematiksel hesaplamalar ve değişken atamaları için kullanılır.	Bir eylemi gerçekleştirmek içindir.
	Eşkenar Dörtgen (Karar)	Bir koşulun kontrol edildiği noktadır. Sonuca göre akış Evet veya Hayır yönünde ilerler.	Yol ayrımında karar vermek içindir.
	Oklar (Akış Yönü)	Adımlar arasındaki ilerleme yönünü gösterir.	İzlenecek yolu gösteren oklardır.

2.2.1. Örnek 1: İki Sayının Toplamını Bulma (Sayfa 25 Örnek 1)

Kullanıcı tarafından girilen iki sayının toplamını bulan örneğin akış şemasını (flowchart) çiziniz. Birinci ve ikinci sayı ve toplam için sırasıyla **s1**, **s2**, **toplam** değişkenleri kullanılacak.

Akış Şeması(Flowchart)

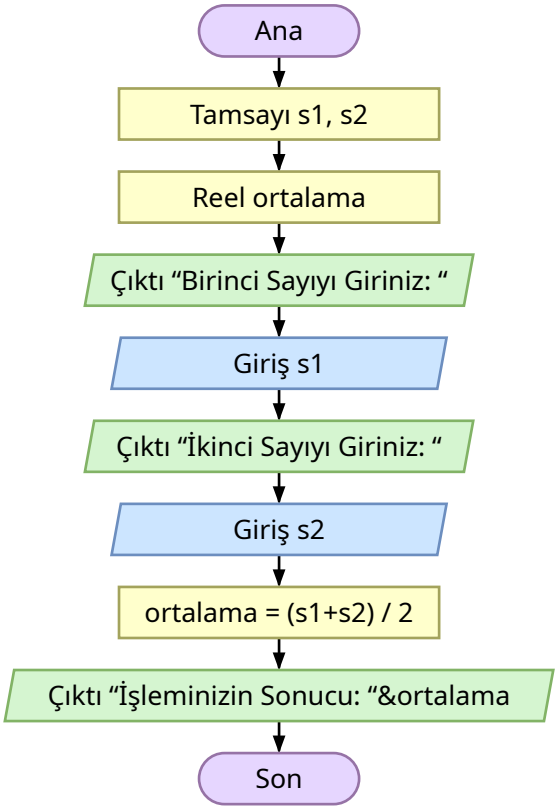


2.2.2. Örnek 2: İki Sayının Ortalamasını Bulma

Kullanıcı tarafından girilen iki sayının ortalamasını bulan örneğin akış şemasını(flowchat) çiziniz. Birinci ve ikinci sayı ve ortalama için sırasıyla **s1**, **s2**, **ortalama** değişkenleri kullanılacak.

Ortalama hesabı: `ortalama = (s1 + s2) / 2`

Akış Şeması(Flowchart)

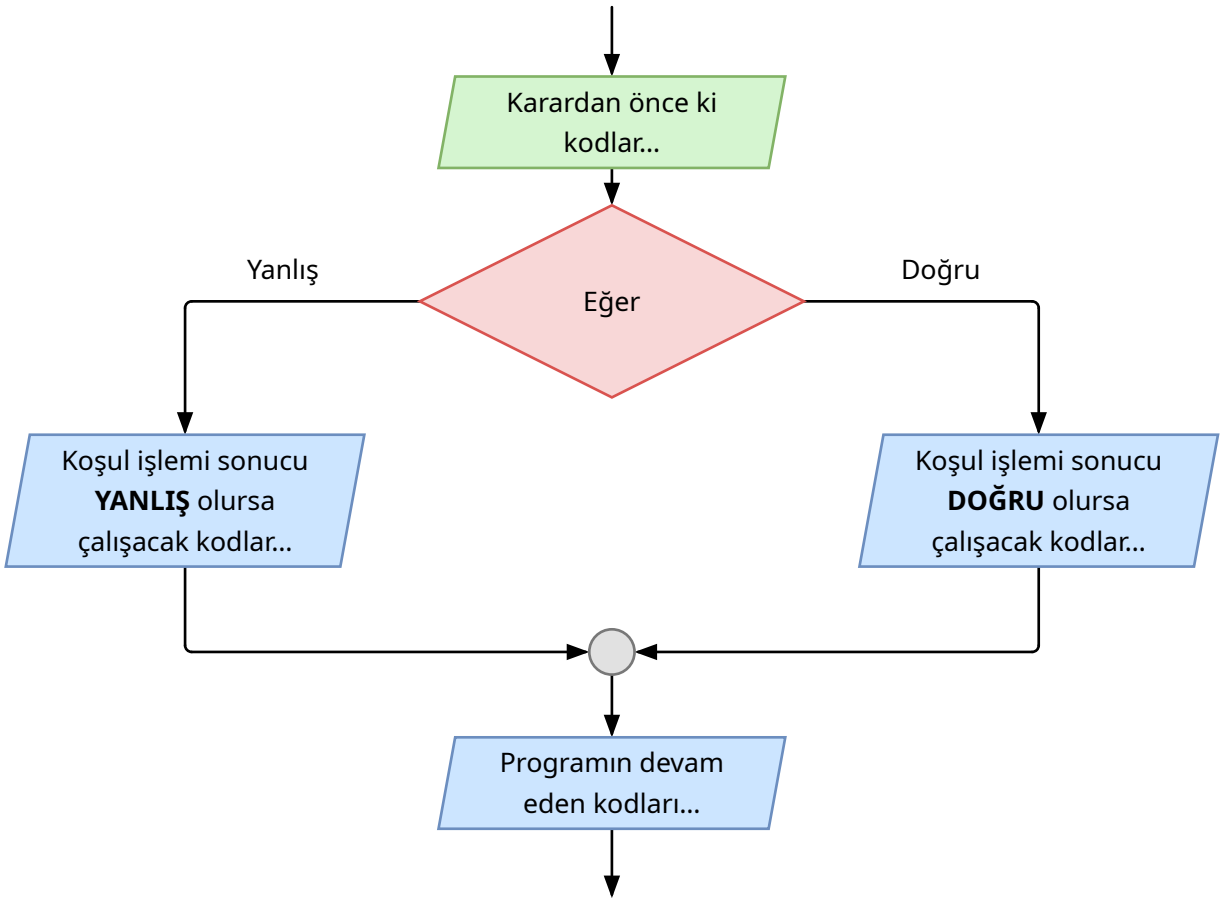


2.2.3. Örnek Çalışma Soruları

1. Yaş Hesaplama: Kullanıcıdan doğum yılını ve mevcut yılı alıp yaşını hesaplayan ve ekrana yazdıran akış şemasını çiziniz.
2. Kare Alan ve Çevre: Bir kenar uzunluğu kullanıcıdan girilen karenin alanını ve çevresini hesaplayıp ekrana yazdıran akış şemasını çiziniz.
3. KDV Hesaplama: Fiyatı girilen bir ürünün %20 KDV tutarını ve KDV dahil toplam satış fiyatını hesaplayıp ekrana yazdıran akış şemasını çiziniz.
4. Birim Dönüşümü (Sıcaklık): Kullanıcıdan Celsius cinsinden sıcaklık değerini alıp Fahrenheit değerine çeviren (F = C *1.8 + 32) ve sonucu gösteren akış şemasını çiziniz.
5. Taksi Ücreti Hesaplama: Taksimetre açılış ücreti 30 TL, kilometre başına ücret 20 TL olan bir şehirde; gidilen kilometre bilgisini kullanıcıdan alıp ödenecek toplam tutarı hesaplayıp yazdıran akış şemasını çiziniz.

2.2.4. Karar Yapıları

Karar yapıları, programın bir koşulu kontrol ederek hangi adımı izleyeceğini belirler. Akış şemasında eşkenar dörtgen ile gösterilir. Koşul doğruysa bir yola, yanlışsa başka bir yola devam edilir. Altta ki şemada görüldüğü gibi eğer işlemi ile yapılan kontrolün sonunda kod doğru ya da yanlış ihtimale doğru hareket edecektir.



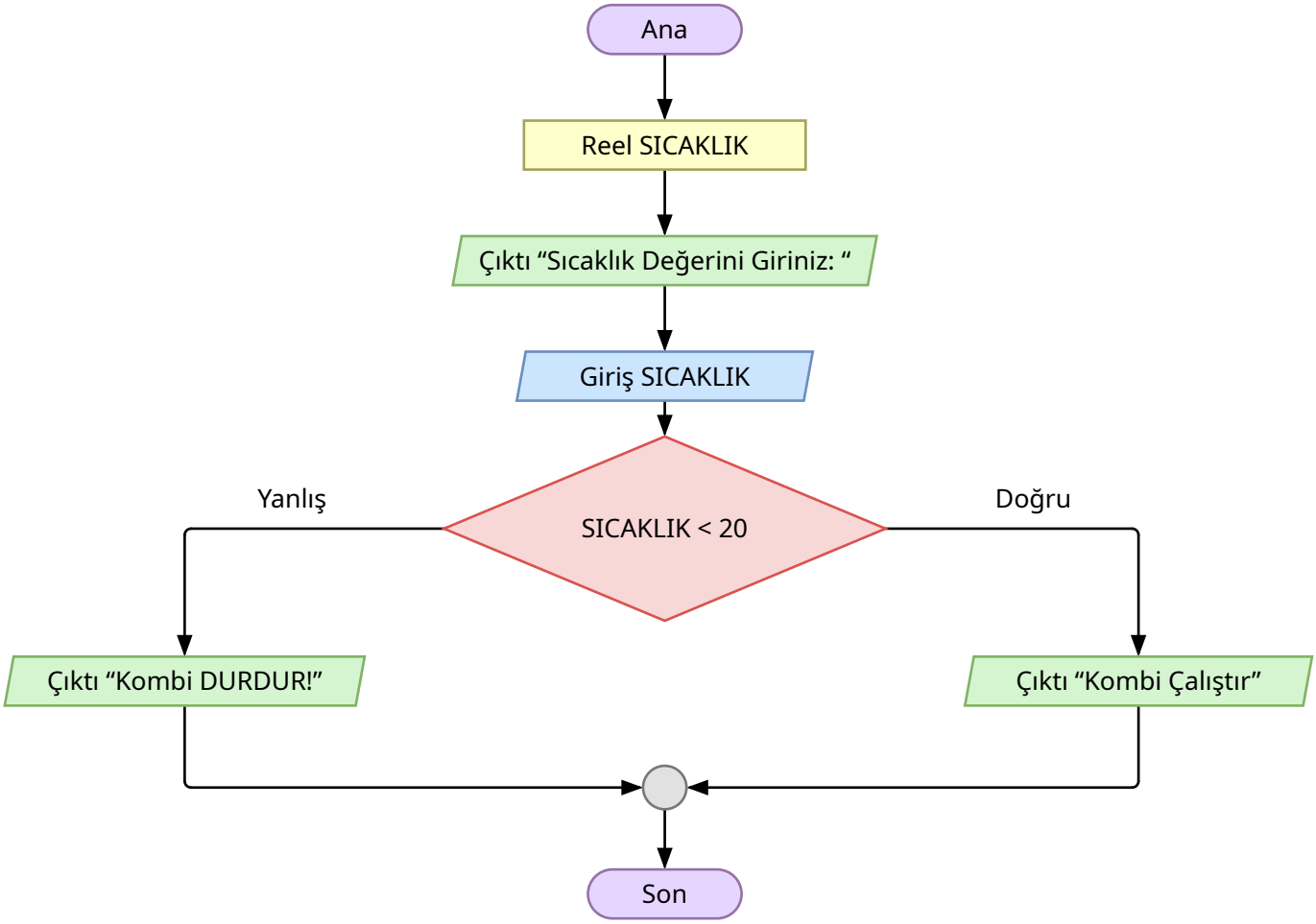
2.2.5. Örnek 3 Sıcaklık Kontrolü

Ortam sıcaklığı (**SICAKLIK**) 20 derecenin altına düştüğünde kombiyi çalıştıran, 20 derecenin üzerine çıktığında ise kombiyi durduran bir termostat programının akış şemasını hazırlayın.

Sözde Kod

1. Adım: BAŞLA
2. Adım: Sıcaklık değeri oku **SICAKLIK**
3. Adım: **EĞER SICAKLIK < 20**
   EVET ise “Kombi Çalıştır”
   HAYIR ise “Kombi Durdu”
4. Adım: BİTİR

Akış Şeması(Flowchart)



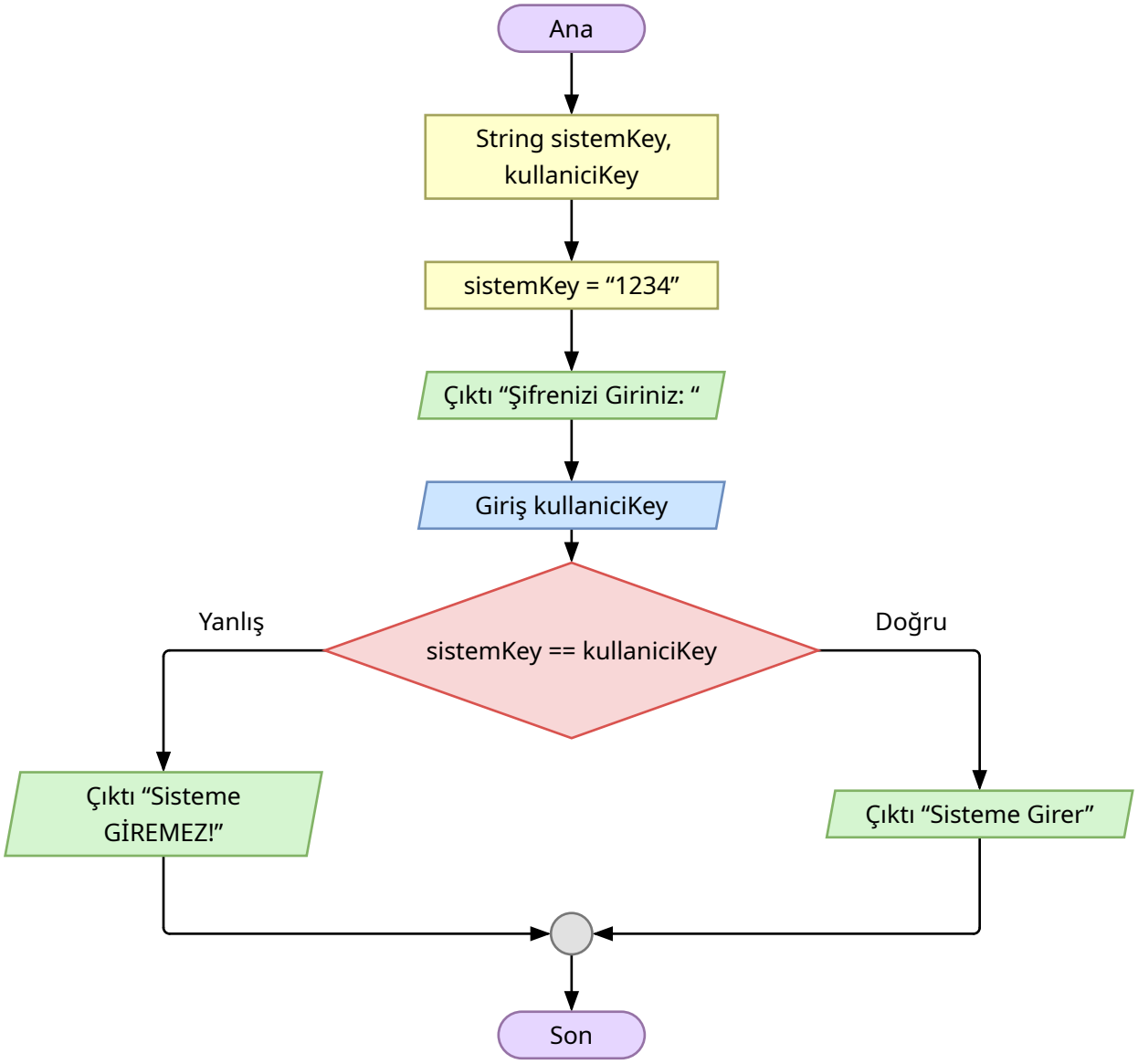
2.2.6. Örnek 4 Hatalı Giriş Kontrolü Algoritması

Kullanıcının önceden belirlediği bir şifre ile sisteme girişini kontrol eden programının akış şemasını hazırlayın. Sistem şifresi için **sistemKey**, kullanıcı şifresi için **kullaniciKey** kullanılacak.

Sözde Kod

1. Adım: BAŞLA
2. Adım: **sistemKey = 1234**
3. Adım: kullanıcıdan (**kullaniciKey**) değerini al
4. Adım: **EĞER sistemKey == kullaniciKey**
   EVET ise “Sisteme Girer”
   HAYIR ise “Sisteme Giremez!”
5. Adım: BİTİR

Akış Şeması(Flowchart)



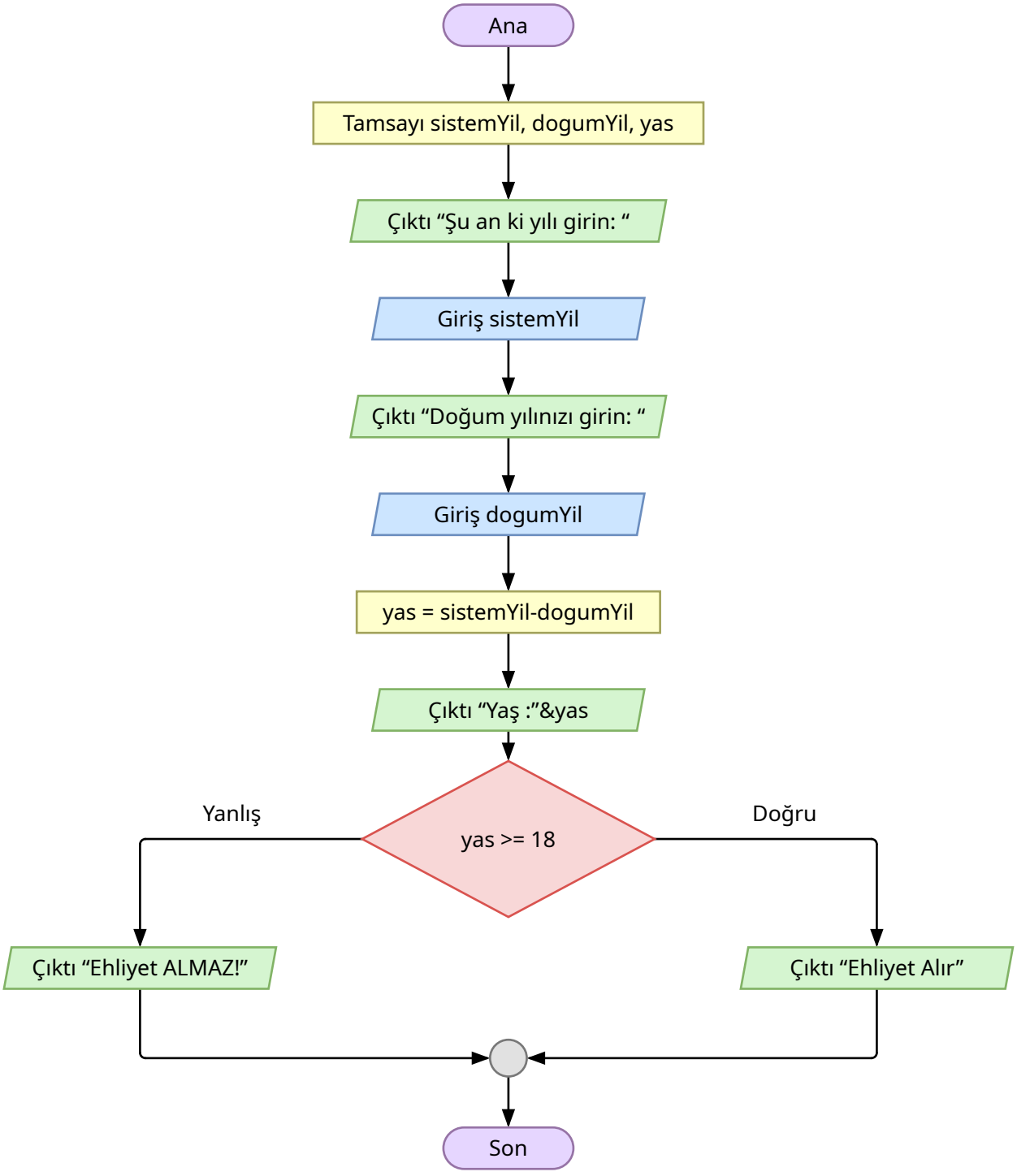
2.2.7. Örnek 5: Yaş Kontrolü ve Ehliyet Alabilme

Kullanıcıdan o günün yılını (**sistemYil**) ve kullanıcıdan doğum yılını (**dogumYil**) alarak kullanıcının 18 yaşından büyük olması durumunda "**Ehliyet Alır!**" küçük olması durumunda "**Ehliyet Almaz!**" yazısını ekrana yazdıran programın sözde kodunu ve elle çalışma tablosunu hazırlayınız.

Sözde Kod

2. Adım: Sistem Tarihi (**sistemYil**) değerini oku.
3. Adım: Kullanıcının Doğum Yılı (**dogumYil**) değerini oku.
4. Adım: **yas = sistemYil - dogumYil** işlemini yap.
5. Adım: **Eğer yas >= 18**
   EVET ise "**Ehliyet Alır!**" mesajını yazdır.
   HAYIR ise "**Ehliyet Almaz!**" mesajını yazdır.

Akış Şeması(Flowchart)



2.2.8. Örnek Çalışma Soruları Karar Yapıları

1. Akıllı Bahçe Sulama Sistemi (Sensör ve Eşik Değeri Mantığı): Bir seradaki toprak nem oranı sensöründen gelen değer **nem** değişkeninde tutulmaktadır.
   - Nem oranı 30'un altına düştüğünde sulama vanasını açan (**"Sulama Başlatıldı"**),
   - 30 veya üzerinde olduğunda ise sulama vanasını kapatan (**"Sulama Durduruldu"**) programın akış şemasını çiziniz.
2. Bakiye ve Ödeme Kontrolü (İki Değişkeni Karşılaştırma): Bir online alışveriş sisteminde kullanıcının mevcut hesap bakiyesi **bakiye**, satın almak istediği sepet tutarı ise **sepet** değişkenidir.
   - Eğer **bakiye** değeri **sepet** tutarına eşit veya büyükse ekrana **"Ödeme Başarılı"** yazdıran ve kalan yeni bakiyeyi ekranda gösteren,
   - Yetersiz ise ekrana **"Yetersiz Bakiye"** uyarısı veren programın akış şemasını çiziniz.
3. Bir öğrencinin vize ve final notunu kullanıcıdan alarak, öğrencinin not ortalamasını hesaplayan programın akış şemasını çizin.
   - Ortalama formülü: ortalama = (vize **0.4) + (final** 0.6)
   - Eğer ortalama değeri 50 veya üzerinde ise ekrana **"Geçti"**, 50'nin altında ise **"Kaldı"** yazdırılacak
4. Web sitesi kargo ücreti hesaplama;
   - Sipariş tutarı 500 TL veya üzerinde ise kargo ücretsizdir
   - Sipariş tutarı 500 TL'nin altında ise 50 TL kargo ücreti eklenmektedir.

   Kullanıcıya son ödenecek toplam tutarı ekrana yazdıran programın akış şemasını hazırlayınız.
5. Kullanıcıdan bir tamsayı (**sayi**) girmesini isteyen, girilen sayının 0'dan büyük olması durumunda ekrana **"Pozitif Sayı"**, 0'dan küçük olması durumunda **"Negatif Sayı"** yazdıran programın akış şemasını hazırlayın.

2.2.9. Döngü(Loop)

Bazı işlemlerin belirli bir koşul sağlanana kadar tekrar edilmesi gerekir. Bu tür işlemler için döngüler kullanılır.

Bir sayaç kontrollü döngü üç temel bölümden oluşur:

- **Başlangıç değeri:** Sayacın başlayacağı değerdir.
- **Döngü koşulu:** Döngünün devam edip etmeyeceğini belirler.
- **Sayaç:** Her tur sonunda değeri artırır veya azaltır.

Koşul doğru olduğu sürece döngü devam eder. Koşul yanlış olduğunda döngü sona erer. Sayaç değerinin değiştirilmesi, döngünün sonsuza kadar sürmesini önler.

2.2.10. Örnek 6 Döngüsel Ekran Yazımı (Sayfa 37 Örnek 8)

Döngü kullanarak ekrana üç kez “Merhaba Geliştirici!” mesajını yazdırınız.

Sözde Kod

1. Adım: **sayac** tam sayı değişkenini tanımla
2. Adım: sayac = 1 ile sayac değişkenine ilk değeri olan biri ata.
3. Adım: Eğer **sayac <= 3** sayac küçük eşit
   EVET ise

   “Merhaba Geliştirici!” yazdız
   sayac = sayac + 1 => sayac değerini bir arttır

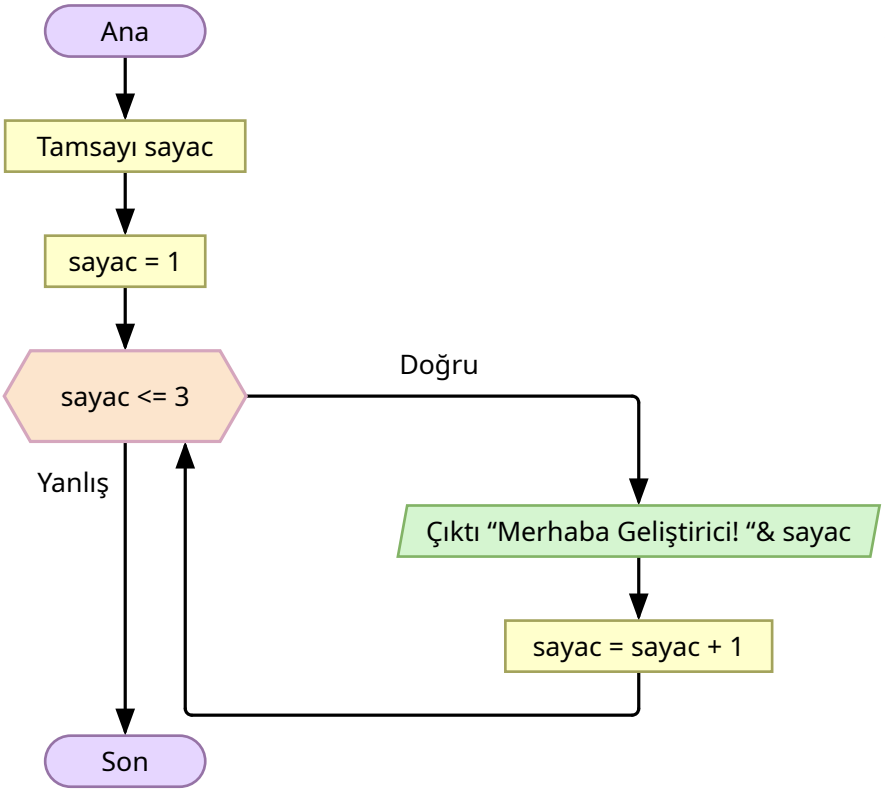
   HAYIR ise

   Döngüden çık

Elle Çalışma Tablosu
Örnek Girdiler: sayac = 3 , **Beklenen Çıktı:** ekrana üç kere “Merhaba Geliştirici” yazdırılacak

Adım	İşlem	sayac	Döngü Koşul (sayac <= 3)	Çıktı	Açıklama
1	Tam sayı **sayac**	-	-	-	sayac değişkeni tanımlanır
2	sayac = 1	1	-	-	sayac değişkenine 1 değeri atanır
3	sayac <= 3	1	1 <= 3 **EVET**	Merhaba Geliştirici	**1<= 3** sonucu **EVET** ekrana yazdırılır sayac 1 arttırılır
4	sayac <= 3	2	2<= 3 **EVET**	Merhaba Geliştirici	**2<= 3** sonucu **EVET** ekrana yazdırılır sayac 1 arttırılır
5	sayac <= 3	3	3<= 3 **EVET**	Merhaba Geliştirici	**3<= 3** sonucu **EVET** ekrana yazdırılır sayac 1 arttırılır
6	sayac <= 3	4	4<= 3 **HAYIR**	-	**4<= 3** sonucu **HAYIR** **Döngü Sonlanır**

Akış Şeması (Flowchart)



Ekran Çıktısı

Merhaba Geliştirici! 1

Merhaba Geliştirici! 2

Merhaba Geliştirici! 3

2.2.11. Örnek 7 Ardışık Sayıların Toplamı (Sayfa 38 Uygulama 4)

Birden başlayarak kullanıcının belirlediği değere kadar belirlenen dahil tüm sayıların toplamını alan kodun akış şemasını hazırlayın. **Örnek Hesap : `n = 3 için toplam = 1 + 2 + 3 = 6`**

Sözde Kod

1. Adım: **sayac, toplam ve n** tam sayı değişkenini tanımla
2. Adım: **sayac = 1** ile sayac değişkenine ilk değeri olan biri ata.
3. Adım: **toplam = 0** başlangıç değerini ata
4. Adım: kullanıcıdan **n** değişkenin değerini al
5. Adım: Eğer **sayac <= n** sayac küçük eşit n
   EVET ise

   toplam = toplam + sayac => toplama işlemini yap
   sayac = sayac + 1 => sayac değerini bir arttır

   HAYIR ise

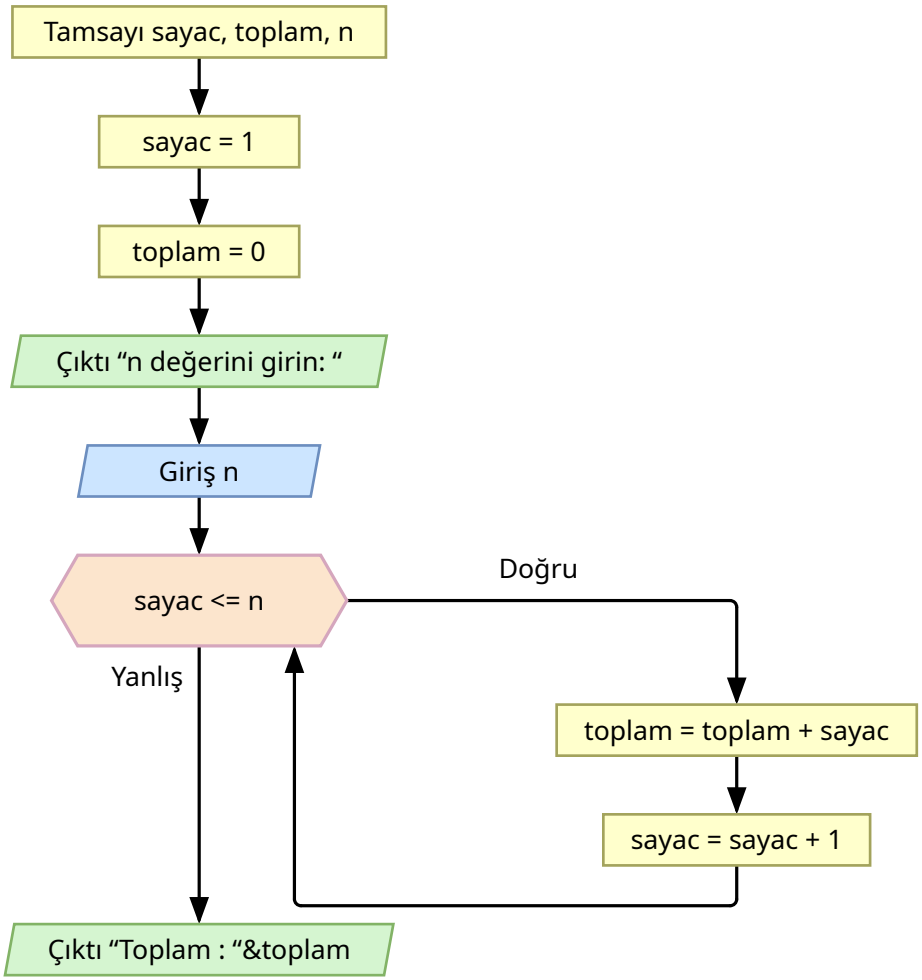
   Döngüden çık
6. Adım: **“Toplam = “&toplam** toplam değerini yazdır.

Elle Çalışma Tablosu

Örnek Girdiler: n = 3 için, **Beklenen Çıktı:** toplam = 6

Ad	İşlem	sayac	toplam	n	Döngü Koşul (sayac <= n)	Çıktı	Açıklama
1	Tam sayı sayac, toplam, n	-	-	-	-	-	Kullanılacak değişkenler tanımlanır
2	sayac = 1, toplam = 0	1	0	-	-	-	sayac ve toplam değişkenleri ilk değerlerini alır
3	n = 3	1	0	3	1 <= 3 **evet**	-	koşul doğru sayac = 2, toplam = 0 + 1 = 1
4	n = 3	2	1	3	2 <= 3 **evet**	-	koşul doğru sayac = 3, toplam =1 + 2 = 3
5	n = 3	3	3	3	2 <= 3 **evet**	-	koşul doğru sayac = 4, toplam = 3 + 3 = 6
6	n = 3	4	6	3	4 <= 3 **HAYIR**	“Toplam :” 6	koşul yanlış sayac = 4, toplam = 6

Akış Şeması (Flowchart)



2.2.12. Örnek Çalışma Soruları Döngü Konusu

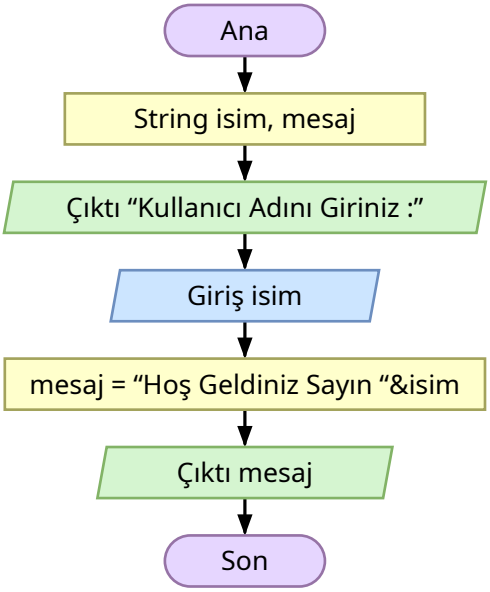
1. 1 ile 100 arasında belirlenen gizli bir sayının (örneğin 33) kullanıcının tahminleriyle bulunmaya çalışıldığı bir tahmin oyununun akış şemasını çiziniz. Döngü kontrolü için boolean tipinde dongu değişkenini kullanınız. Girilen tahmin gizli sayıdan farklı olduğu sürece kullanıcıyı “Sayı Küçük!” veya “Sayı Büyük!” şeklinde yönlendiriniz. Her tahmini sayarak doğru sonuca ulaşıldığında toplam deneme sayısını ekrana yazdırınız.
2. Geri Sayım ve Fırlatma (Geriye Doğru Sayıcı Mantığı): Bir roket fırlatma simülasyonu için 10′dan başlayıp 1′e kadar birer birer geriye doğru sayan ve her sayıyı ekrana yazdıran, döngü bittiğinde ise ekrana “Fırlatma Başarılı!” mesajı veren programın akış şemasını çiziniz.
   İpucu: Döngü sayacını her adımda 1 azaltınız (SAYAC = SAYAC - 1).
3. Çift Sayıların Çarpımı (Faktöriyel / Kümülatif Çarpım Mantığı): Kullanıcıdan pozitif bir çift sayı (N) alarak, 2′den başlayıp N değerine kadar olan (N dahil) tüm çift sayıların çarpımını hesaplayan programın akış şemasını ve sözde kodunu hazırlayınız.
   Örnek Hesap: N = 6 için Carpim = 2 * 4 * 6 = 48
   Öğrenci Notu: Çarpım için başlangıç etkisiz elemanını (CARPIM = 1) ve adım artışını (SAYAC = SAYAC + 2) doğru belirleyiniz.
4. Sınıf Not Ortalaması Hesaplama (Döngü İçinde Girdi Alma ve Ortalama): Kullanıcıdan önce sınıfta kaç öğrenci olduğu bilgisini (ogrenciAdet) alan, ardından döngü yardımıyla bu adet kadar öğrenci notunu (ogrenciNot) teker teker kullanıcıdan isteyip toplayan, döngü sonunda ise sınıfın genel not ortalamasını hesaplayıp ekrana yazdıran programın akış şemasını çiziniz.
   Elle Çalışma: ogrenciAdet = 3 ve girilen notlar 70, 80, 90 olacak şekilde elle çalışma (izleme) tablosunu oluşturunuz.
5. PIN Kodu ile Telefon Kilidi Açma (3 Hak Sınırlı): Telefonun kayıtlı PIN kodu dogruPin = 1234 olarak belirlenmiştir. Kullanıcıya en fazla 3 deneme hakkı tanınacaktır.
   Kullanıcı doğru PIN’i girerse ekrana “Kilit Açıldı” yazıp döngü derhal sonlanmalıdır.
   Kullanıcı 3 hakkında da hatalı giriş yaparsa ekrana “Telefon Kilitlendi!” uyarısı verilmelidir.
   Bu senaryoyu gerçekleştiren programın akış şemasını çiziniz.
6. Sıfır Girilene Kadar Pozitif Sayıları Sayma: Kullanıcı klavyeden 0 (sıfır) girene kadar sürekli sayı girişi kabul eden bir program tasarlayınız.
   Girilen sayılar arasından yalnızca pozitif olanların kaç adet olduğunu saymalıdır (pozitifAdet).
   Kullanıcı 0 girdiğinde döngüden çıkılmalı ve toplamda kaç adet pozitif sayı girildiği ekrana yazdırılmalıdır.

2.3. Algoritmalarda Metinsel Veriler ve İşlemler

Bilgisayarlar, sayılar dışında isim ve şifre gibi metinsel verileri de işler. Çift tırnak içinde yazılan metinlere **`string`** (**karakter dizisi**) denir. Metinsel veriler doğrudan matematiksel işlemlerde kullanılamaz.

2.3.1. Örnek Metin Birleştirme (Sayfa 42 Örnek 9)

Kullanıcıdan aldığı isim bilgisiyle ekrana “Hoş Geldiniz Sayın :” + isim şeklinde yazı yazdıran kodun akış şemasını çiziniz?



2.4. Dijital Depo

Yazılım geliştirme sürecinde kaynak kodların güvenle saklandığı, sürümlerinin takip edildiği alanlara dijital depo (repository) denir. Günümüzde GitHub ve GitLab gibi platformlar; projelerin sürümler halinde yönetilmesini, dağıtılmasını ve ekiplerce ortak geliştirilmesini sağlar. Her deponun vitrini sayılan README.md dosyası ise projenin amacını ve kullanımını özetleyen teknik imzasıdır.

Sizlerin yapacağı yazılım projeleri de ilerleyen derslerimizde GitHub üzerinde yayımlanarak paylaşıma açılacaktır.

Ahmet Tuna POTUR GitHub Hesabı

Geliştirdiğim açık kaynaklı yazılımları, eğitim materyallerini ve projelerin sürüm geçmişlerini incelemek için kişisel GitHub profilimi ziyaret edebilirsiniz.

A.Tuna POTUR GitHub

Bağlantıya tıklayın
github.com/tunapotur

Programlamanın Temelleri 9 GitHub Projesi

Ders notlarımızın kaynak kodlarını barındıran örnek dijital depoyu buradan inceleyebilirsiniz. Projenin amacını öğrenmek için vitrin belgesi olan README.md dosyasını ve geliştirme sürecindeki sürüm adımlarını gözlemleyebilirsiniz.

GitHub Prog. Temelleri

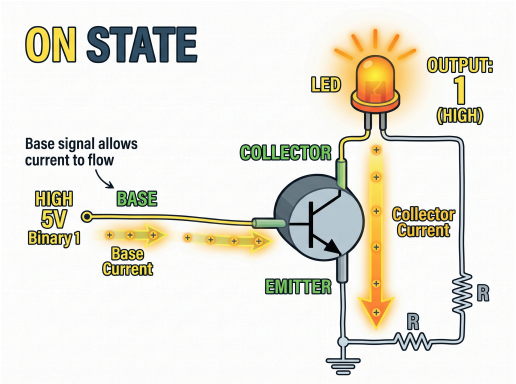
Bağlantıya tıklayın
kisa.link/WeLYU

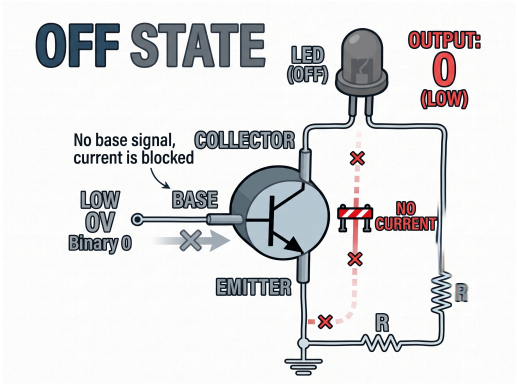
II Temel Programlama Yapıları

1. Donanımın Temeli: Transistör ve İkili Sistem (Binary)

Transistör elektrik akımını geçirerek ya da keserek 1 veya 0 değerini üreten bilgisayarın en temel donanım birimidir. İşlemci, hafıza gibi bilgisayar donanımlarında milyarlarca transistör bulunur.

Bir Transistör Elektrik Akını Geçirir veya Keser





- Elektrik Akımı Var (Açık) : 1 (Bit)
- Elektrik Akımı Yok (Kapalı) : 0 (Bit)

1 ve 0 durumunu oluşturan en küçük bilgi birimine **Bit (Binary Digit)** denir. Bilgisayar dünyasındaki tüm yazılımlar, videolar, oyunlar ve metinler fiziksel temelde sadece 1 ve 0'lardan ibarettir.

Yazılım tarihinin en başında, bilgisayarlara bir iş yaptırmak için işlemcinin tek doğal dili olan Makine Dili (**İkilik Sistem / Binary**) kullanılıyordu. Programcılar, komutları `1011 0000` gibi 0 ve 1 dizileri halinde delikli kartlara veya anahtarlara işliyordu. Ancak bu yöntem insan gözü için son derece yorucuydu ve tek bir rakamın yanlış yazılması tüm programı bozuyordu.

2. İlk Bilgisayarlar ve 8 Bitlik Komut Girişleri (Altair 8800)

Erken dönem **Altair 8800** gibi bilgisayarların klavyesi veya ekranı yoktu. Ön panelinde sıra sıra fiziksel anahtarlar (**toggle switches**) ve LED lambalar yer alıyordu. Bu bilgisayar, sadece 1 ve 0 değerleri kullanılarak donanıma nasıl komut verildiğine ve hesap yaptırıldığına somut bir örnektir.

```
+-------------------------------------------------------------+
|                  ALTAIR 8800 ÖN PANELİ                      |
|                                                             |
|   LED'ler (Çıkış):    (O) (X) (O) (O)   (X) (X) (O) (X)     |
|                        0   1   0   0     1   1   0   1      |
|                                                             |
|   Anahtarlar (Giriş): [v] [^] [v] [v]   [^] [^] [v] [^]     |
|                        0   1   0   0     1   1   0   1      |
+-------------------------------------------------------------+
```

- Programcılar her bir 8 bitlik komutu anahtarları tek tek yukarı (1) ve aşağı (0) indirerek belleğe elle yüklüyordu.
- İki sayıyı toplamak için bile onlarca 8 bitlik komutu hatasız girmek gerekiyordu. Bu süreç son derece yavaş, zihinsel olarak yorucu ve hata yapmaya açıktı.

Tarihi Eser Bilgisayarı Programladım | Altair 8800

Ömer Bektaş'ın kanalında Altair 8800 hakkında detaylı bir video bulunmakta.

www.youtube.com/watch?v=1zwTRA0yaYo

Altair 8800 Emulatorü

Altair 8800 cihazını denemek istiyorsanız tarayıcınızda kullanabileceğiniz Altair 8800 Emulator linki.

www.s2js.com/altair/sim.html

Ömer Bektaş'ın Youtube Kanalı

Ömer Bektaş, bilişim dünyasına yönelik çok faydalı içerikler üreten bir YouTube içerik üreticisidir. Bu kanalı takip etmeniz sizin için çok faydalı olacaktır.

www.youtube.com/@merbekta

3. Onaltılık Sayı Sistemi (Hexadecimal)

İkilik Sistemde oluşan zorlukları aşmak için **Onaltılık (Hexadecimal)** Sayı Sistemi kullanılmaya başlandı. Dört adet ikilik (binary) basamağın tek bir hexadecimal karaktere (0-9 ve A-F arası) karşılık gelmesi sayesinde, `1011 0000`, `0110 0001` gibi karmaşık bir makine kodu `B0`, `61` şeklinde çok daha kısa ve okunabilir bir biçimde ifade edildi. Bu yöntem yazımı kısaltıp hata payını azaltsa da, programcıların hâlâ hangi sayı kodunun hangi işlemi yaptığını ezbere bilmesi gerekiyordu.
$2^4 = 16$ olduğu için 4 bitlik bir binary sayı tam olarak bir hexadecimal basamağı karşılık gelir.

4-Bit İkilik (Binary) ve Onaltılık (Hexadecimal) Karşılıklar Tablosu

Hex	Dec	Binary	Hex	Dec	Binary	Hex	Dec	Binary	Hex	Dec	Binary
0	0	0000	**4**	4	0100	**8**	8	1000	**C**	12	1100
1	1	0001	**5**	5	0101	**9**	9	1001	**D**	13	1101
2	2	0010	**6**	6	0110	**A**	10	1010	**E**	14	1110
3	3	0011	**7**	7	0111	**B**	11	1011	**F**	15	1111

Hex: Hexadecimal(16'lık), **Dec:** Decimal(10'luk), **Binary:** Binary(4 Bit),

4. Metin Temsili: 8 Bit = 1 Byte ve ASCII Tablosu

Bir harfi veya sembolü sayılarla temsil etmek için standart referans tablosu **ASCII (American Standard Code for Information Interchange)** geliştirilmiştir. Böylelikle en temel düzeyde gerekli olan karakterlere için **ASCII** tablosundaki karakterin sayısal bir karşılığı oluşturuldu.

- Bilgisayar mimarisinin temelinde 1 karakter 1 Byte (8 bit) yer kaplar. ASCII tablosunda 256 karakter bulunmasının sebebi 8 bit'in yan yana getirilerek 1 Byte oluşturmasıdır. 1 Byte aynı zamanda iki hexadecimal karakterle de kolaylıkla ifade edilebilir.
- **Kontrol Karakterleri:** Tablodaki 0 ile 31 arasındaki ilk 32 karakter ve 127 numaralı `DEL` karakteri ekranda basılabilir bir harf veya sembol değildir. Bunlar cihazlar arası iletişimi ve imleç hareketlerini yöneten **Kontrol Karakterleri (Control / Non-printable Characters)** olarak adlandırılır.

ASCII Code - Karakter Kodlama Tablosu

Tüm ASCII karakterlerini incelemek için bu referans tablosunu kullanabilirsiniz.

www.ascii-code.com

Temel ASCII Tablosu (32-127 Karakterin Kodu)

Dec	Hex	Binary	Chr	Dec	Hex	Binary	Chr	Dec	Hex	Binary	Chr	Dec	Hex	Binary	Chr
32	20	0010 0000	*SP*	56	38	0011 1000	8	80	50	0101 0000	P	104	68	0110 1000	h
33	21	0010 0001	!	57	39	0011 1001	9	81	51	0101 0001	Q	105	69	0110 1001	i
34	22	0010 0010	"	58	3A	0011 1010	:	82	52	0101 0010	R	106	6A	0110 1010	j
35	23	0010 0011	#	59	3B	0011 1011	;	83	53	0101 0011	S	107	6B	0110 1011	k
36	24	0010 0100	$	60	3C	0011 1100	<	84	54	0101 0100	T	108	6C	0110 1100	l
37	25	0010 0101	%	61	3D	0011 1101	=	85	55	0101 0101	U	109	6D	0110 1101	m
38	26	0010 0110	&	62	3E	0011 1110	>	86	56	0101 0110	V	110	6E	0110 1110	n
39	27	0010 0111	'	63	3F	0011 1111	?	87	57	0101 0111	W	111	6F	0110 1111	o
40	28	0010 1000	(	64	40	0100 0000	@	88	58	0101 1000	X	112	70	0111 0000	p
41	29	0010 1001	)	65	41	0100 0001	A	89	59	0101 1001	Y	113	71	0111 0001	q
42	2A	0010 1010	*	66	42	0100 0010	B	90	5A	0101 1010	Z	114	72	0111 0010	r
43	2B	0010 1011	+	67	43	0100 0011	C	91	5B	0101 1011	[	115	73	0111 0011	s
44	2C	0010 1100	,	68	44	0100 0100	D	92	5C	0101 1100	\	116	74	0111 0100	t
45	2D	0010 1101	-	69	45	0100 0101	E	93	5D	0101 1101	]	117	75	0111 0101	u
46	2E	0010 1110	.	70	46	0100 0110	F	94	5E	0101 1110	^	118	76	0111 0110	v
47	2F	0010 1111	/	71	47	0100 0111	G	95	5F	0101 1111	_	119	77	0111 0111	w
48	30	0011 0000	0	72	48	0100 1000	H	96	60	0110 0000	`	120	78	0111 1000	x
49	31	0011 0001	1	73	49	0100 1001	I	97	61	0110 0001	a	121	79	0111 1001	y
50	32	0011 0010	2	74	4A	0100 1010	J	98	62	0110 0010	b	122	7A	0111 1010	z
51	33	0011 0011	3	75	4B	0100 1011	K	99	63	0110 0011	c	123	7B	0111 1011	{
52	34	0011 0100	4	76	4C	0100 1100	L	100	64	0110 0100	d	124	7C	0111 1100	|
53	35	0011 0101	5	77	4D	0100 1101	M	101	65	0110 0101	e	125	7D	0111 1101	}
54	36	0011 0110	6	78	4E	0100 1110	N	102	66	0110 0110	f	126	7E	0111 1110	~
55	37	0011 0111	7	79	4F	0100 1111	O	103	67	0110 0111	g	127	7F	0111 1111	*DEL*

5. Bit'ten Terabayt'a Veri Büyüklükleri

Bilgisayar dünyasındaki tüm içeriklerin fiziksel temelde sadece 1 ve 0'lardan ibaret olduğunu ve 1 ve 0 durumun oluşturan en küçük bilgi birimine **Bit (Binary Digit)** dendiğini söylemiştik.

- **Bit (Binary Digit):** Bilgisayardaki en küçük mantıksal veri birimidir. Sadece **0** veya **1** değerini alabilir. Devredeki tek bir transistör lamba anahtarı gibi düşünülebilir: ya kapalıdır (0) ya da açıktır (1).
- **Byte:** 8 adet bitin yan yana gelmesiyle oluşan veri kümesidir. $8\ \text{Bit} = 1\ \text{Byte}$

Örneğin **"A"** harfi bellekte **1 Byte** yer kaplar gibi düşünebiliriz.

- Günlük hayatta metrik sistemde birimler onluk sisteme (10^3) göre biner biner büyür (1 kg = 1000 g, 1 km = 1000 m). Ancak bilgisayarlar **2'lik sayı sistemine (Binary)** göre çalışır. Bilgisayar mimarisinde bellek adreslemesi 2'nin üsleri şeklinde katlanarak ilerler. Onluk sistemdeki 1000 sayısına ikili tabanda en yakın kuvvet: 1024 sayısıdır ($2^{10} = 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 1024$).
  Bu nedenle bilişim mimarisinde her bir büyüklük basamağı bir öncekinin **1024 katı** (2^{10}) kabul edilir.

Veri Büyüklüğü Basamakları

Birim	Kapasite	Kısaltma	Somut Karşılığı
Bit	0 veya 1	**b**	Tek bir transistör / elektrik sinyali
Byte	$8\ \text{Bit}$	**B**	Tek bir karakter (harf, rakam, sembol)
Kilobyte	$1024\ \text{Byte}$	**KB**	Düz metinden oluşan 1 sayfalık belge
Megabyte	$1024\ \text{KB}$	**MB**	1 dakikalık MP3 ses kaydı veya telefon fotoğrafı
Gigabyte	$1024\ \text{MB}$	**GB**	Yaklaşık 1 saatlik HD kalitede film
Terabyte	$1024\ \text{GB}$	**TB**	Geniş harici yedekleme diski, çok sayıda büyük oyun
Petabyte	$1024\ \text{TB}$	**PB**	Büyük veri merkezleri veya bulut sunucu havuzları
Exabyte	$1024\ \text{PB}$	**EB**	Küresel internet trafiğinin aylık ölçekteki hacmi

Donanım Üreticisi vs. İşletim Sistemi Farkı: Piyasadan satın alınan **1TB** kapasiteli bir SSD veya harici disk bilgisayara takıldığında, depolama alanı genellikle yaklaşık **931 GB** görünür. Bu durum donanım hatası değildir:

Donanım Üreticileri **Onluk** sayı birimini kullanarak $1 \ \text{KB} = 10^3 \ \text{Byte}$ şeklinde hesap yapar:

$$1 \ \text{TB} = 1.000.000.000.000 \ \text{Byte}$$

Fakat işletim sistemleri ikilik sayı sistemine göre ölçüm yaptığı için altta ki bölme işleminin sonucu satın aldığınız diskinizin gerçek boyutu **1TB** altında olacaktır.

$$\frac{1.000.000.000.000}{1024 \times 1024 \times 1024} \approx 931.32 \ \text{GB}$$

Sizce bu disk üreticilerinin yaptığı bir hile midir?

6. Makine Kodundan Assembly Diline

Altair 8800 gibi erken dönem bilgisayarlarda, işlemciye komut vermek için sürekli **`1` ve `0`** dizilimlerinden oluşan makine kodlarını kullanmak oldukça zahmetliydi ve hatalara açıktı. Makine kodlarını daha kısa ve okunabilir biçimde ifade etmek için **Onaltılık (Hexadecimal) sayı sistemi** kullanılmaya başlandı. Ancak hexadecimal gösterim, makine kodunu daha kolay yazmayı sağlasa da komutların anlamını doğrudan açıklamıyordu.

Bu nedenle makine kodundaki komutları daha kolay hatırlayabilmek ve yazabilmek için İngilizce kelimelerin kısaltmalarından oluşan sembolik isimler (**Mnemonic**) kullanılmaya başlandı. Bu sembolik komutları kullanan dile **Assembly dili** adı verildi. Böylece programcılar, yalnızca **`1` ve `0`** dizilimleriyle uğraşmak yerine, daha anlaşılır ve hatırlanması kolay komutlarla program yazabilir hâle geldi.

Hexadecimal Yerine Assembly Kullanımı Örneği

Hexadecimal	Yerine	Assembly	Açıklama
`B0 61`	⟶	`MOV AL, 61H`	AL yazmacına **`61H`** verisi taşındı.
`04 05`	⟶	`ADD AL, 05H`	AL yazmacındaki değere **`05H`** eklendi.
`2C 02`	⟶	`SUB AL, 02H`	AL yazmacındaki değerden **`02H`** çıkarıldı.

Bu sembolik komutlarla yazılan programlar, **Assembler (Çevirici)** adı verilen bir program aracılığıyla işlemcinin anlayabileceği makine koduna dönüştürüldü.

x86 Assembly: İki Sayıyı Toplama Örneği

```
section .text
global _start

_start:
    mov eax, 5          ; ilk sayı
    mov ebx, 7          ; ikinci sayı
    add eax, ebx        ; eax = eax + ebx  (5 + 7 = 12)

    mov ebx, eax        ; sonucu ebx'e al (exit kodu ebx'ten okunur)
    mov eax, 1          ; sys_exit sistem çağrısı numarası
    int 0x80            ; çekirdeği çağır
```

Dillerin Atası | Assembly

Ömer Bektaş'ın kanalında Assembly dili hakkında detaylı bir video bulunmakta.

www.youtube.com/watch?v=yWXuG3RLkTE

7. Programlama Dilleri

Her ne kadar **`1` ve `0`** veya **hexadecimal sayılar** ile kodlama yapmaya göre **Assembly** dili büyük bir kolaylık sağlasa da projeler büyüdükçe kodları yazmak zorlaşıyordu. Ayrıca Assembly kodları işlemciden işlemciye farklılık gösteriyor, bu da her yeni işlemci için kodların yeniden yazılması zorunluluğunu doğuruyordu. Bu duruma **donanım bağımlılığı** deniyordu.

İşte bu sorunu çözmek için **Fortran, COBOL ve C** gibi programlama dilleri ortaya çıktı. Artık iki sayıyı toplamak için işlemci yazmaçlarıyla uğraşmak yerine doğrudan **`sonuc = a + b`** yazmak yeterli hale geldi. Programlama dilleri ile yazılan kodlar, **Derleyici (Compiler)** veya **Yorumlayıcı (Interpreter)** adı verilen araçlarla makine koduna çevrilerek işlemci üzerinde çalıştırılmaktadır. Ayrıca programlama dili ile yazılan tek bir kodun hedef bilgisayarda derlenmesiyle **donanım bağımlılığı** sorunu çözüldü ve aynı kodların farklı işlemci mimarilerinde çalışabilmesi sağlandı.

Derleyici (Compiler): Programcının yazdığı kodun tamamını önce baştan sona okur, yazım hatalarını kontrol eder ve tüm kodu bir bütün olarak makine diline çevirir. Kodun yazımında (söz diziminde) bir hata varsa derleme işlemi tamamlanmaz.

Yorumlayıcı (Interpreter): Kodu bir bütün olarak değil satır satır okuyarak anında çalıştırır. Bir satır çevrilip çalıştırıldıktan sonra sıradaki satıra geçilir. Hata tespiti açısından yorumlayıcı, hatanın bulunduğu satıra gelene kadar programı çalıştırır ve o noktada durur. Bu yaklaşım tıpkı bir tercümanın, konuşmacının cümlelerini anlık olarak çevirmesine benzer. **Python ve Ruby**, bu yaklaşımı kullanan dillere örnek gösterilebilir.

Evrim Aşaması	Temsil Biçimi	Örnek Kod	Temel Avantajı	Temel Zorluğu
1. Makine Dili (Binary)	`0` ve `1` dizileri	`1011 0000` `0110 0001`	İşlemci hiçbir çeviriye ihtiyaç duymadan doğrudan çalıştırır.	İnsan tarafından okunması ve yazılması çok zordur.
2. Hexadecimal (16'lık Sistem)	`0-9` ve `A-F` karakterleri	`B0 61`	Uzun 0 ve 1 dizilerini 4'lü gruplar halinde kısaltarak yazım kolaylığı sağlar.	Komutların sayısal karşılıklarını akılda tutmak gerekiyordu.
3. Assembly Dili (Sembolik Dil)	Sembolik kısaltmalar *(Mnemonics)*	`MOV AL, 61h`	Komutlar (`MOV`, `ADD`, `SUB`) anlamlı kelimelerle ifade edilir.	Her işlemci mimarisi için farklıdır; büyük projelerde kod çok uzar.
4. Yüksek Seviyeli Diller	Günlük dil ve matematiksel ifadeler	`int a = 97;`	Öğrenmesi kolaydır, donanımdan bağımsızdır ve hızlı yazılım geliştirmeyi sağlar.	İşlemcide çalışabilmesi için derleyici veya yorumlayıcı ile çevrilmesi gerekir.

Örnek Python Kodu

Kullanıcıdan bir değer alıp bu değere 25 ekleyerek sonucu ekrana yazdıran kod örneği.

```python
puan = int(input("Puan Giriniz: "))
print("Puanı:", puan)
puan = puan + 25
print("Güncellenmiş Puan:", puan)
```

Python Ekran Çıktısı

```
Puan Giriniz: 50
Puanı: 50
Güncellenmiş Puan: 75
```

Bilgisayarın Dili

Transistörden yüksek seviyeli programlama dillerine: 1 ve 0'ın yolculuğu

Programlamanın Evrimi

1 Makine Dili

`1011 0000`

✓ İşlemci çeviri olmadan çalıştırır
✗ Okuması ve yazması çok zor

2 Hexadecimal

`B0 61`

✓ 0/1 dizilerini 4'lü gruplarla kısaltır
✗ Komut kodları ezberlenmeli

3 Assembly

`MOV AL, 61H`

✓ Anlamlı kısaltmalar (MOV, ADD)
✗ Her işlemciye özel, kod uzar

4 Yüksek Seviyeli

`sonuc = a + b`

✓ Kolay, donanımdan bağımsız, hızlı
✗ Derleyici/yorumlayıcı gerekir

1 · Transistör: Akımı Geçirir veya Keser

Bilgisayarın en temel donanım birimi; işlemci ve hafızada **milyarlarca** transistör bulunur.

AÇIK · Çıkış 1
Base: HIGH (5V)
Akım geçer, LED yanar

KAPALI · Çıkış 0
Base: LOW (0V)
Akım kesilir, LED söner

2 · Bit (Binary Digit)

1 ve 0 durumunu oluşturan **en küçük bilgi birimi.** Yazılımlar, videolar, oyunlar ve metinler fiziksel olarak yalnızca 1 ve 0'lardan oluşur.

1 0 1 1

3 · Altair 8800 (1970'lerin ortası)

Klavye ve ekran yoktu; ön panelde **toggle switch** ve LED'ler vardı. Her 8 bitlik komut anahtarlarla tek tek belleğe yükleniyordu: yukarı = 1, aşağı = 0.

```
+-------------------------------------------------------------+
|                  ALTAIR 8800 ÖN PANELİ                      |
|                                                             |
|   LED'ler (Çıkış):    (O) (X) (O) (O)   (X) (X) (O) (X)     |
|                        0   1   0   0     1   1   0   1      |
|                                                             |
|   Anahtarlar (Giriş): [v] [^] [v] [v]   [^] [^] [v] [^]     |
|                        0   1   0   0     1   1   0   1      |
+-------------------------------------------------------------+
```

İki sayıyı toplamak için bile onlarca komut hatasız girilmeliydi: yavaş, yorucu, hataya açık.

4 · Hexadecimal: 4 Bit = 1 Basamak

$2^4 = 16$ olduğundan 4 bit, tek bir hex karakterine (0–9, A–F) karşılık gelir.

0 (0) 0000	**1** (1) 0001	**2** (2) 0010	**3** (3) 0011
4 (4) 0100	**5** (5) 0101	**6** (6) 0110	**7** (7) 0111
8 (8) 1000	**9** (9) 1001	**A** (10) 1010	**B** (11) 1011
C (12) 1100	**D** (13) 1101	**E** (14) 1110	**F** (15) 1111

`1011 0000` → **B0** `0110 0001` → **61**

5 · Metin Temsili: 8 Bit = 1 Byte ve ASCII

Harf ve sembolleri sayılarla temsil etmek için **ASCII** standardı geliştirildi. 1 karakter = **1 Byte** (8 bit) yer kaplar ve iki hex karakterle yazılır.

0 1 0 0 0 0 0 1

1 Byte = hex **41** = onluk **65** = '**A**'

Chr	Dec	Hex	Binary
SP	32	20	0010 0000
0	48	30	0011 0000
A	65	41	0100 0001
a	97	61	0110 0001
DEL	127	7F	0111 1111

0–31 ve 127	32–47	48–57	65–90	97–122
Kontrol karakterleri (basılamaz)	Boşluk ve semboller	Rakamlar 0–9	Büyük harfler A–Z	Küçük harfler a–z

6 · Makine Kodundan Assembly'ye

Komutlar, İngilizce kelime kısaltmalarından oluşan sembolik isimlerle (**mnemonic**) yazıldı. **Assembler** bu kodu makine koduna çevirir.

Hex	Assembly	Açıklama
B0 61	MOV AL, 61H	AL'ye 61H taşındı
04 05	ADD AL, 05H	AL'ye 05H eklendi
2C 02	SUB AL, 02H	AL'den 02H çıkarıldı

x86 · 5 + 7

```
mov eax, 5 ; ilk sayı
mov ebx, 7 ; ikinci sayı
add eax, ebx ; 12
Yüksek seviyede: sonuc = a + b
```

7 · Programlama Dilleri: Derleyici ve Yorumlayıcı

Fortran, COBOL ve C gibi dillerde çıktı; makine koduna çevrilir. Öğrenmesi kolaydır, donanımdan bağımsızdır ve hızlı yazılım geliştirmeyi sağlar.

Derleyici (Compiler)

Kodun **tamamını** baştan sona okur, hataları denetler, bütünü makine diline çevirir. Söz diziminde hata varsa derleme tamamlanmaz.

Yorumlayıcı (Interpreter)

Kodu **satır satır** okuyup anında çalıştırır; hatalı satıra gelince durur. Tıpkı anlık çeviri yapan bir tercüman gibi (Python, Ruby).

PYTHON · YORUMLAYICI ÖRNEĞİ

```
1 puan = int(input("Puan Giriniz: "))
2 print("Puanı:", puan)
3 puan = puan + 25
4 print("Güncellenmiş Puan:", puan)
```

Satırlar sırayla yorumlanıp çalıştırılır.

8. Değişkenler ve Bellek(RAM)

Değişken; programın çalışması sırasında verilerin bellekte (RAM) geçici olarak tutulduğu alanlara verilen isimdir. Kullanıcıdan veya harici bir kaynaktan alınan bilgiler bu değişkenlerde saklanır; kod çalıştıkça hesaplamalarda kullanılır ve süreç içinde yeni değerler alabilir.
Değişkenler yardımıyla bellekte (RAM) saklanan verilere kolayca erişilir; kod çalıştırıldığında bu değişken isimleri çağrılarak temsil ettikleri veriler üzerinde işlemler gerçekleştirilir.

Örnek Python Kodu Kullanıcıdan alınan tam sayı değerine 25 ekleyerek sonucu ekrana yazdıran kod örneği

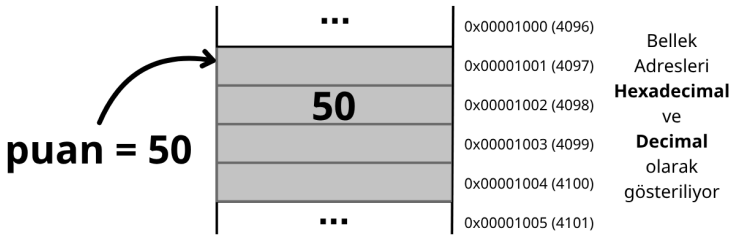
```python
puan = int(input("Puan Giriniz: "))
print("Puanı:", puan)
puan = puan + 25
print("Güncellenmiş Puan:", puan)
```

Python Ekran Çıktısı

```
Puan Giriniz: 50
Puanı: 50
Güncellenmiş Puan: 75
```

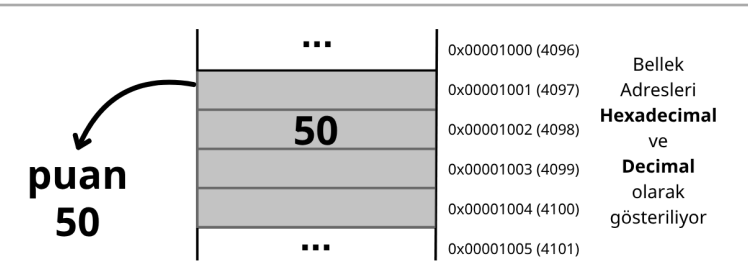
1. Değer Atama
Giriş yapılan **50** değeri, RAM'de **puan** ismiyle bir adrese kaydedilir.

```
Puan Giriniz: 50
```

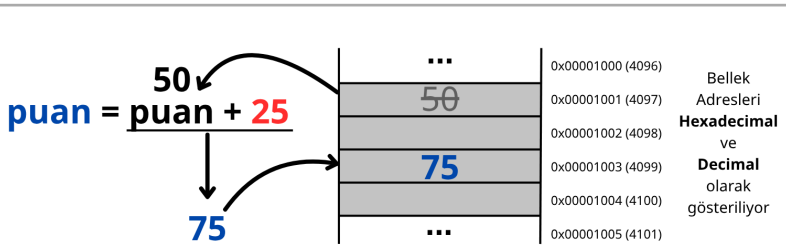


2. Hafızadan Çağırma
puan değişkeni RAM'deki adresinden çağrılarak **50** değeri ekrana yazdırılır.

```
Puanı: 50
```

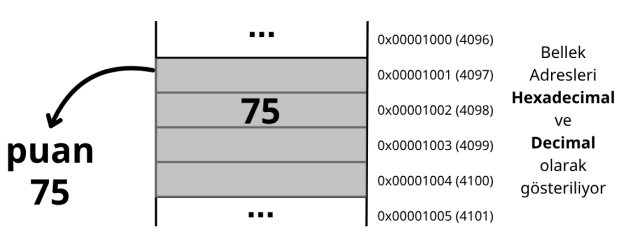


3. Değer Güncelleme
Mevcut **50** değeri hafızadan alınır, **25** eklenir ve elde edilen yeni **75** değeri tekrar **puan** değişkenine atanır.



4. Güncel Hali Çağırma
Güncellenen **puan** değişkeni hafızadan çağrılarak ekrana yazdırılır.

```
Güncellenmiş Puan: 75
```



Değişkenler ve Bellek

Programda tanımlanan değişkenler, geçici olarak bellekte (RAM) bir alan tutar ve bu alanlara değerler kaydedilir.

Python Kodu

```
1  puan = int(input("Puan Giriniz: "))
2  print("Puanı:", puan)
3  puan = puan + 25
4  print("Güncellenmiş Puan:", puan)
```

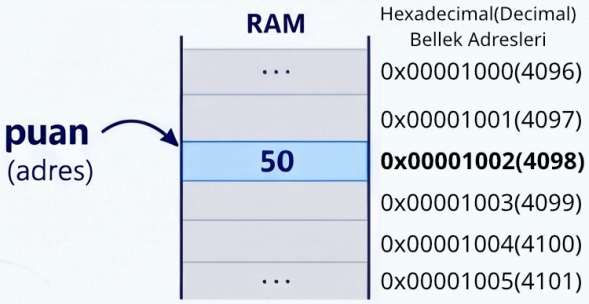
Kullanılan Değişken

puan
(tamsayı / int)
Puan değerini geçici olarak tutan değişken.

1 Değer Atama

Giriş yapılan **50** değeri, RAM'de **puan** ismiyle bir adrese kaydedilir.

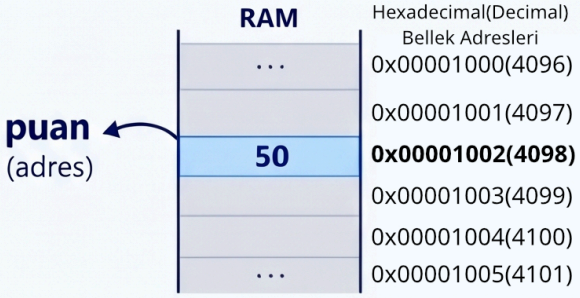
```
Puan Giriniz: 50
```



2 Hafızadan Çağırma

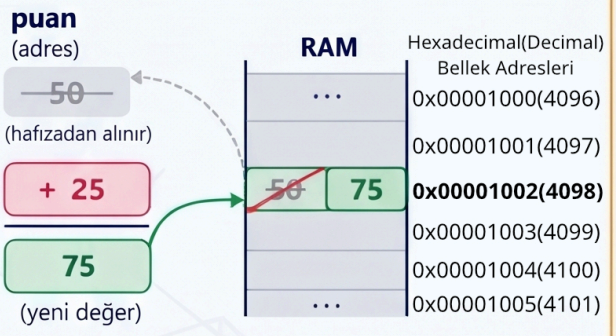
puan değişkeni RAM'deki adresinden çağırılarak **50** değeri ekrana yazdırılır.

```
Puanı: 50
```



3 Değer Güncelleme

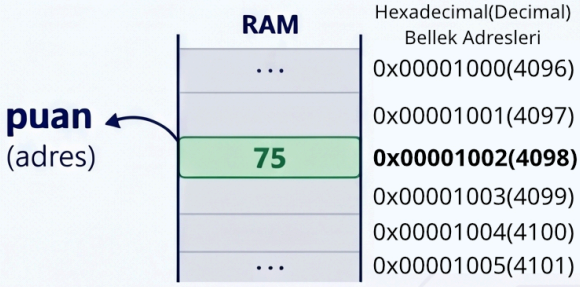
Mevcut **50** değeri hafızadan alınır, **25** eklenir ve elde edilen yeni **75** değeri tekrar **puan** değişkenine atanır.



4 Güncel Hali Çağırma

Güncellenen **puan** değişkeni hafızadan çağırılarak ekrana yazdırılır.

```
Güncellenmiş Puan: 75
```

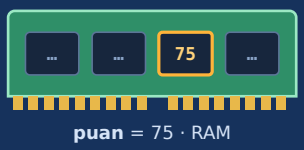


Kısaca: Değişkenler, değerleri geçici olarak RAM'de tutan etiketlerdir. Program çalıştığı sürece bu değerler kullanılabilir ve güncellenebilir.

x = 50
RAM

Değişkenler ve Bellek

Programda tanımlanan değişkenler, geçici olarak bellekte (RAM) bir alan tutar ve bu alanlara değerler kaydedilir.



puan = 75 · RAM

Python Kodu

```
1  puan = int(input("Puan Giriniz: "))
2  print("Puanı:", puan)
3  puan = puan + 25
4  print("Güncellenmiş Puan:", puan)
```

Kullanılan Değişken

puan

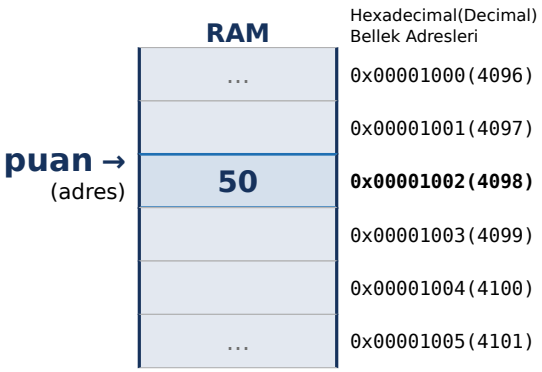
(tamsayı / int)

Puan değerini geçici olarak tutan değişken.

1 Değer Atama

Giriş yapılan **50** değeri, RAM'de **puan** ismiyle bir adrese kaydedilir.

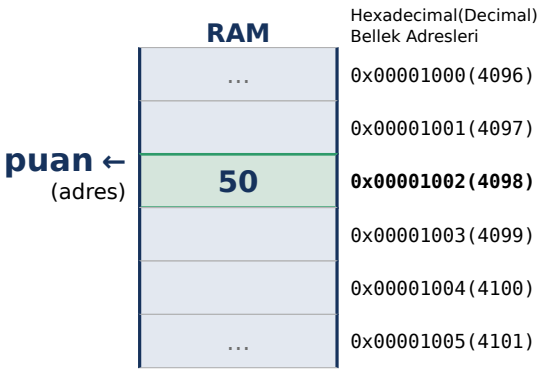
```
Puan Giriniz: 50
```



2 Hafızadan Çağırma

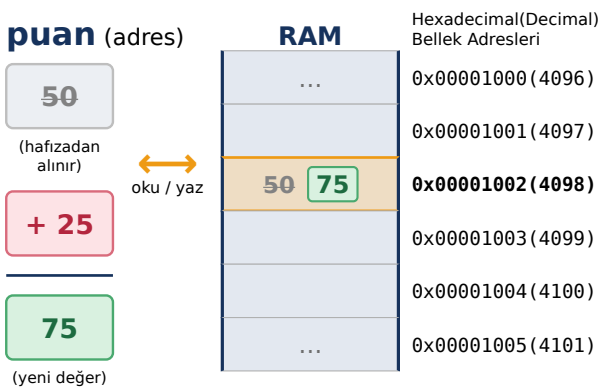
puan değişkeni RAM'deki adresinden çağırılarak **50** değeri ekrana yazdırılır.

```
Puanı: 50
```



3 Değer Güncelleme

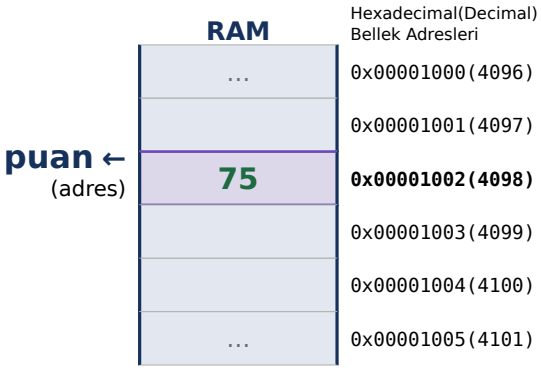
Mevcut **50** değeri hafızadan alınır, **25** eklenir ve elde edilen yeni **75** değeri tekrar **puan** değişkenine atanır.



4 Güncel Hali Çağırma

Güncellenen **puan** değişkeni hafızadan çağırılarak ekrana yazdırılır.

```
Güncellenmiş Puan: 75
```



Kısaca: Değişkenler, değerleri geçici olarak RAM'de tutan etiketlerdir. Program çalıştığı sürece bu değerler kullanılabilir ve güncellenebilir.

9. Veri Türleri ve RAM Kullanımı

Bilgisayar programları verileri RAM (bellek) üzerinde saklar. RAM'i, üst üste dizilmiş çok sayıda küçük kutu gibi düşünebiliriz. Her kutunun boyutu 1 bayt'tır (1 bayt = 8 bit; bit yalnızca 0 ya da 1 tutabilir).

- Bu veri **ne tür** bir veri? (sayı mı, karakter mi, doğru/yanlış mı?)
- Bu yüzden RAM'de **kaç kutu (bayt)** ayrılmalı ve bu veriyle **hangi işlemler** yapılabilir?

> Temel veri türleri konusunu anlatmak için C++ dili ve 16-bit sistemler (Arduino Uno, Turbo C) esas alınmıştır. Bu tercih, veri türlerinin donanımsal mantığının daha rahat anlaşılması amacıyla yapılmıştır. Derslerimizde ise Python programlama dili kullanılacak olup, Python'da veri türleri ve bellek yapısı farklı şekilde işlemektedir.

9.1. Boolean (bool): doğru / yanlış

- RAM'de en az yer kaplayan veri türüdür: **1 bayt**.
- Yalnızca iki değer alabilir: **true** (doğru, 1) veya **false** (yanlış, 0).
- Karar verme (if, while) işlemlerinde kullanılır.

```cpp
bool kapiAcik = true;
bool sinavGecti = false;
bool girisYapti = true;
```

bool — **1 byte**

```cpp
bool kapiAcik = true;
```

1. byte: 0 0 0 0 0 0 0 **1**

true değeri 1, olarak saklanır.

bool — **1 byte**

```cpp
bool sinavGecti = false;
```

1. byte: 0 0 0 0 0 0 0 0

false değeri 0 olarak saklanır.

> Aslında doğru/yanlış bilgisi tek bir bit ile saklanabilir; ancak RAM'e ulaşım bayt bayt yapıldığı için bilgisayar bool için de 1 tam bayt ayırır.

9.2. Karakter (char)

- RAM'de 1 bayt yer kaplar.
- Tek bir karakter tutar: harf, rakam karakteri, noktalama işareti.
- Tek tırnak içinde yazılır: 'A'.
- Bilgisayar karakterleri ASCII tablosundaki sayı karşılığı ile saklar (örneğin 'A' karakteri 65).
- 1 bayt ($8\text{bit} = 2^{16} = 256$), ASCII tablosundaki 256 karakter için yeterlidir.
- Karakterler metin için kullanılır, matematiksel işlemlere girmez. '5' karakteri ile 5 sayısı aynı şey değildir.

```cpp
char harf = 'A';
char rakamKarakteri = '5'; // bu bir sayı değil, karakterdir
char cinsiyet = 'E'; // 'E' (Erkek) veya 'K' (Kadın) gibi tek harfli kısaltmalar
char paraBirimi = '$'; // Özel semboller ve işaretler
char islemTuru = '+'; // Matematiksel operatör sembolleri
char cevapSikki = 'C'; // Çoktan seçmeli testlerde işaretlenen şık
```

char — **1 byte**

```cpp
char harf = 'A';
```

1. byte: 0 **1** 0 0 0 0 0 **1**

'A' karakterinin ASCII kodu 65'tir.

char — **1 byte**

```cpp
char karakter = '5';
```

1. byte: 0 0 **1** **1** 0 **1** 0 **1**

'5' karakterinin ASCII kodu 53'tür Bellekte 1 byte kaplar.

9.3. Tam Sayı (int)

- RAM'de **2 bayt** (16 bit) yer kaplar.
- 16 bit ile $2^{16} = 65536$ farklı değer saklanabilir.
- Yalnızca tam sayıları tutar.
- Toplama, çıkarma, çarpma, bölme ve mod(kalan) işlemlerinde kullanılır.

```c
int a = 7;
int b = 2;
int toplam = a + b // toplam = 9
int sonuc = a / b; // sonuc = 3 (3.5 değil!)
int kalan = a % b; // kalan = 1
```

int — 2 byte

```c
int puan = 50;
```

1. byte	0	0	0	0	0	0	0	0
2. byte	0	0	**1**	**1**	0	0	**1**	0

50 = 32 + 16 + 2 olduğu için son byte 00110010 olur. İlk byte'ın tamamı 0'dır.

int — 2 byte

```c
int sayi = 12377;
```

1. byte	0	0	**1**	**1**	0	0	0	0
2. byte	0	**1**	0	**1**	**1**	0	0	**1**

12377 = 8192 + 4096 + 64 + 16 + 8 + 1 olduğu için ilk byte 00110000 (12288), son byte ise 01011001 (89) olur.

9.4. Ondalıklı Sayı (float)

- RAM'de **4 bayt** yer kaplar; **int**'ten 2 kat fazla alan.
- Virgüllü sayıları tutar. Kodda nokta ile yazılır: 3.14.
- Tam sayı ve küsurat kısımlarını ayrı işler.
- Küsuratlı ve geniş aralıklı sayılarda kullanılır.
- Tüm matematiksel işlemlerde kullanılabilir.

```c
float a = 7;
float b = 2;
float sonuc = a / b; // => 3.5
float pi = 3.14;
float alan = pi * b * b; // => 12.56
```

float — 4 byte

```c
float pi = 3.14;
```

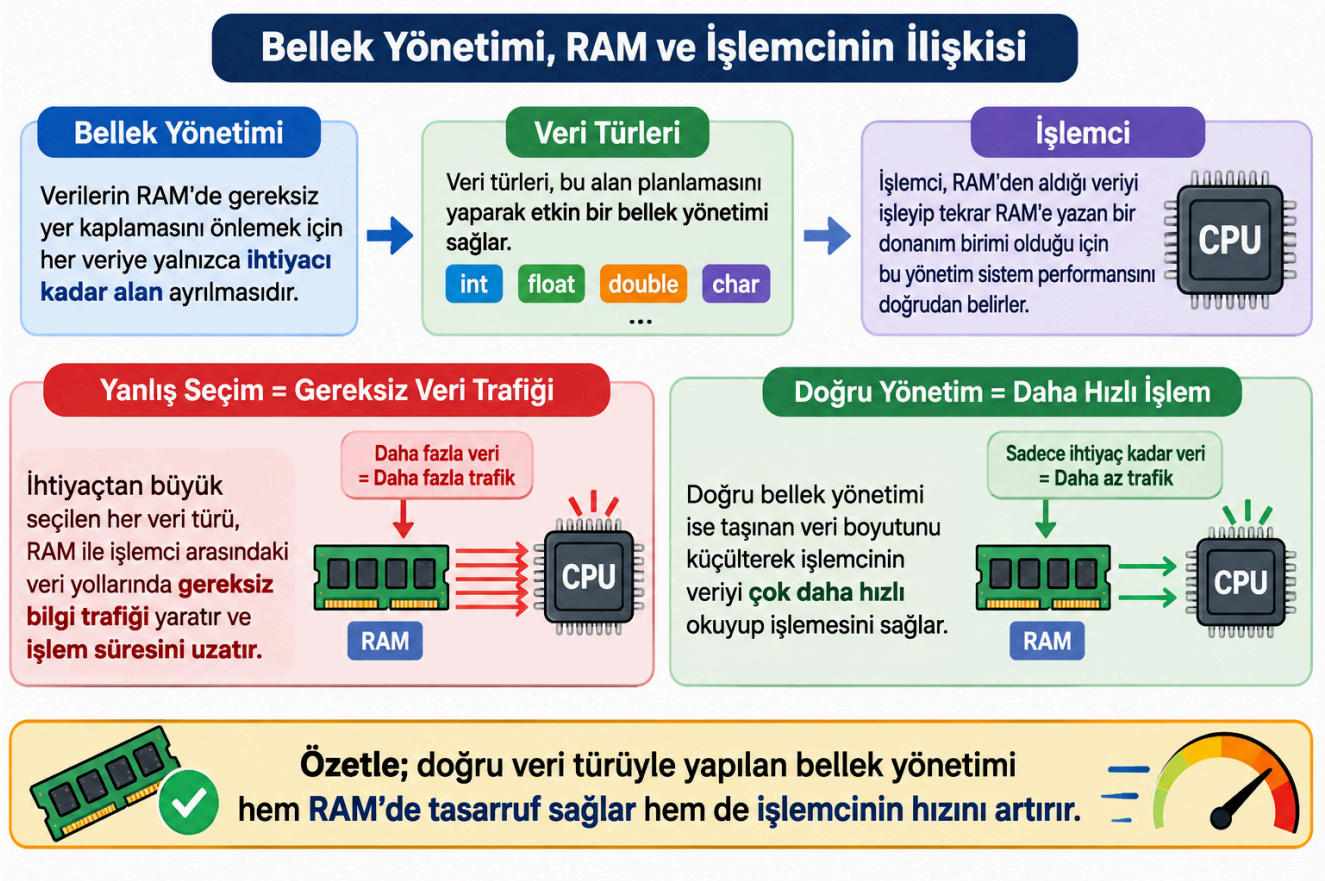
1. byte	0	**1**	0	0	0	0	0	0
2. byte	0	**1**	0	0	**1**	0	0	0
3. byte	**1**	**1**	**1**	**1**	0	**1**	0	**1**
4. byte	**1**	**1**	0	0	0	0	**1**	**1**

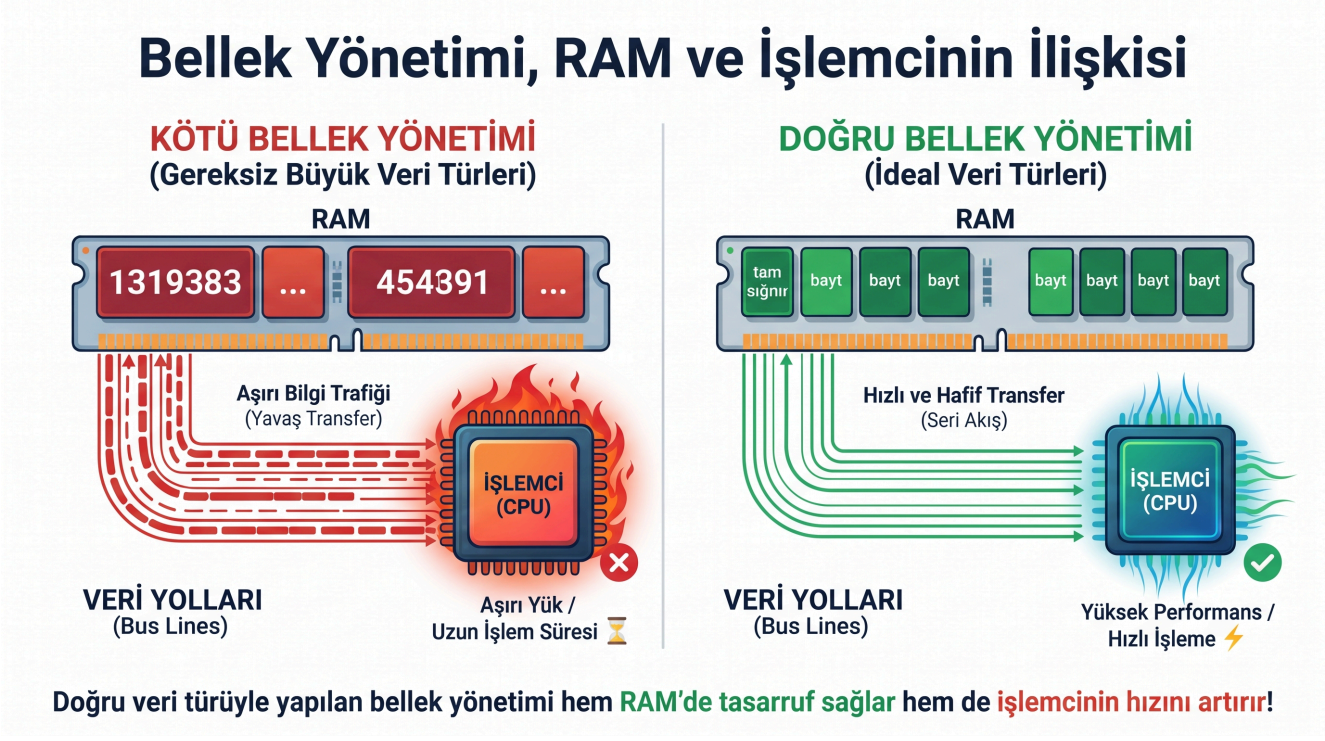
Ondalıklı sayılar IEEE 754 biçiminde saklanır ve 4 byte yer kaplar.

Ne zaman int, ne zaman float?
float her işlemde kullanılabilir; ancak hem daha fazla RAM kaplar hem de işlemciyi daha çok yorar. Bu yüzden **gerekmedikçe int** yerine **float** kullanılmamalıdır. Küsurat gerekiyorsa (ortalama, not, fiiyat, sıcaklık gibi) float tercih edilir.

10. Bellek Yönetimi, RAM ve İşlemcinin İlişkisi

Bellek yönetimi, verilerin RAM'de gereksiz yer kaplamasını önlemek için her veriye yalnızca ihtiyacı kadar alan ayrılmasıdır. Veri türleri, bu alan planlamasını yaparak etkin bir bellek yönetimi sağlar. İşlemci, RAM'den aldığı veriyi işleyip tekrar RAM'e yazan bir donanım birimi olduğu için bu yönetim sistem performansını doğrudan belirler. İhtiyaçtan büyük seçilen her veri türü, RAM ile işlemci arasındaki veri yollarında gereksiz bilgi trafiği yaratır ve işlem süresini uzatır. Doğru bellek yönetimi ise taşınan veri boyutunu küçülterek işlemcinin veriyi çok daha hızlı okuyup işlemesini sağlar. **Özetle; doğru veri türüyle yapılan bellek yönetimi hem RAM'de tasarruf sağlar hem de işlemcinin hızını artırır.**





11. Karakterden Metne (Karakter Dizisi)

Tek bir **char** verisi bellekte yalnızca **1 Bayt** yer kaplar ve tek başına sadece bir karakteri tutabilir. Birden fazla karakterden oluşan bir kelimeyi veya metni bellekte saklamak istediğimizde ise tekil karakterler yan yana getirilerek bir **karakter dizisi** oluşturulur. Programlamada aynı türdeki verilerin bellekte peş peşe sıralanarak tek bir grup halinde tutulmasına **dizi (array)** adı verilir. Metinler de aslında karakterlerin yan yana dizildiği birer dizidir; bu nedenle aşağıdaki yapı ile her bir harf için RAM üzerinde art arda gelen (bitişik) 1 baytlık çekmeceler tahsis edilir.

> **Toplam Bellek Alanı = Karakter Sayısı x 1 Byte**

KEMAL Metni RAM üzerinde nasıl saklanır?

char — **5 byte**

```
char isim[5] = {'K', 'E', 'M', 'A', 'L'};
```

1. byte	0	**1**	0	0	**1**	0	**1**	**1**
2. byte	0	**1**	0	0	0	**1**	0	**1**
3. byte	0	**1**	0	0	**1**	**1**	0	**1**
4. byte	0	**1**	0	0	0	0	0	**1**
5. byte	0	**1**	0	0	**1**	**1**	0	0

Her bir harf ASCII karşılığı ile bellekte 1 byte yer kaplar (K:75, E:69, M:77, A:65, L:76). Toplam 5 karakter = 5 byte.

Ada Lovelace Metni RAM üzerinde nasıl saklanır?

char — **12 byte**

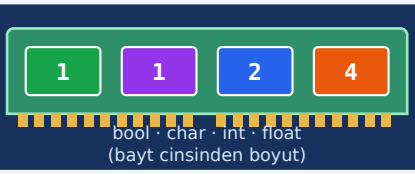
```
char tam_isim[12] = {'A', 'd', 'a', ' ', 'L', 'o', 'v', 'e', 'l', 'a', 'c', 'e'};
```

1. byte	0	**1**	0	0	0	0	0	**1**
2. byte	0	**1**	**1**	0	0	**1**	0	0
3. byte	0	**1**	**1**	0	0	0	0	**1**
4. byte	0	0	**1**	0	0	0	0	0
5. byte	0	**1**	0	0	**1**	**1**	0	0
6. byte	0	**1**	**1**	0	**1**	**1**	**1**	**1**
7. byte	0	**1**	**1**	**1**	0	**1**	**1**	0
8. byte	0	**1**	**1**	0	0	**1**	0	**1**
9. byte	0	**1**	**1**	0	**1**	**1**	0	0
10. byte	0	**1**	**1**	0	0	0	0	**1**
11. byte	0	**1**	**1**	0	0	0	**1**	**1**
12. byte	0	**1**	**1**	0	0	**1**	0	**1**

Boşluk karakteri (' ') de bellekte 1 byte yer kaplar (ASCII: 32). Toplam 12 karakter = 12 byte.

Veri Türleri ve RAM Kullanımı

Programlar verileri RAM'de bayt kutularında saklar. Verinin türü, kaç bayt yer ayrılacağını ve hangi işlemlerin yapılabileceğini belirler.



RAM: üst üste dizilmiş küçük kutular

1 bayt	1 bayt	1 bayt	1 bayt	1 bayt	1 bayt	1 bayt	1 bayt

▼ yakınlaştır

0	1	0	0	0	0	0	1

1 bayt = 8 bit (her bit yalnızca 0 ya da 1)

1. Bu veri **ne tür** bir veri? (sayı mı, karakter mi, doğru/yanlış mı?)
2. RAM'de **kaç kutu (bayt)** ayrılmalı ve **hangi işlemler** yapılabilir?

bool — 1 bayt

- Yalnızca iki değer: **true** (1) ya da **false** (0)
- Karar yapılarında kullanılır (if, while)

```
bool kapiAcik = true;
```

1. bayt	0	0	0	0	0	0	0	1

Tek bit yeter; ancak RAM'e bayt bayt ulaşıldığı için 1 tam bayt ayrılır.

char — 1 bayt

- Tek karakter: harf, rakam karakteri, noktalama
- ASCII sayısıyla saklanır: 'A' = 65
- Matematiksel işlem yapılmaz: '5' ≠ 5

```
char harf = 'A';
```

1. bayt	0	1	0	0	0	0	0	1

1 bayt, ASCII tablosundaki 256 karakter için yeterlidir.

int — 2 bayt

- 2^{16} = 65536 değer: −32768 ... +32767
- Yalnızca tam sayı: 7 / 2 = 3, 7 % 2 = 1
- Toplama, çıkarma, çarpma, bölme, mod

```
int puan = 50;
```

1. bayt	0	0	0	0	0	0	0	0
2. bayt	0	0	1	1	0	0	1	0

50 = 32 + 16 + 2 olduğu için son bayt 00110010 olur; ilk baytın tamamı 0'dır.

float — 4 bayt

- Virgüllü sayılar: 3.14 (kodda nokta ile)
- int'in 2 katı yer; 7 / 2 = 3.5 (küsuratlı)
- Çok daha geniş aralık, IEEE 754 biçimi

```
float pi = 3.14;
```

1. bayt	0	1	0	0	0	0	0	0
2. bayt	0	1	0	0	1	0	0	0
3. bayt	1	1	1	1	0	1	0	1
4. bayt	1	1	0	0	0	0	1	1

Ondalıklı sayılar IEEE 754 biçiminde 4 bayt yer kaplar.

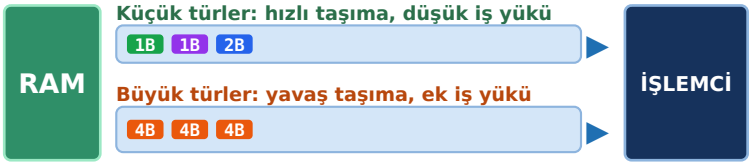
Karakterden Metne

Toplam bellek alanı = karakter sayısı × 1 bayt

Her harf için RAM'de art arda (bitişik) 1 baytlık kutular ayrılır. `"Ada Lovelace"`: 12 karakter (boşluk dahil) = **12 bayt**.

A	d	a	␣	L	o	v	e	l	a	c	e
65	100	97	32	76	111	118	101	108	97	99	101

RAM ve İşlemci İlişkisi



Büyük veri türleri hem RAM'de daha fazla yer kaplar hem de işlemciyi daha fazla meşgul eder.

int mi, float mı?

`int` Tam sayı yetiyorsa

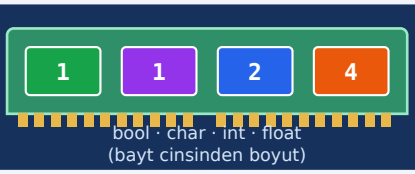
`float` Küsurat gerekiyorsa (ortalama, not, fiyat, sıcaklık)

Gerekmedikçe int yerine float kullanılmamalıdır: daha fazla RAM kaplar, işlemciyi daha çok yorar.

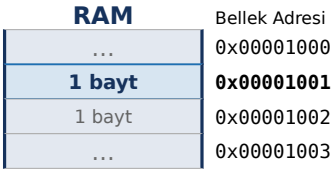
Not: Anlatımda C++ ve 16 bit sistemler (Arduino Uno, Turbo C) esas alınmıştır; böylece veri türlerinin donanımsal mantığı daha rahat anlaşılır. Derslerimizde Python kullanılacaktır; Python'da veri türleri ve bellek yapısı farklı işler.

Veri Türleri ve RAM Kullanımı

Programlar verileri RAM'de bayt kutularında saklar. Verinin türü, kaç bayt yer ayrılacağını ve hangi işlemlerin yapılabileceğini belirler.



RAM: üst üste dizilmiş küçük kutular





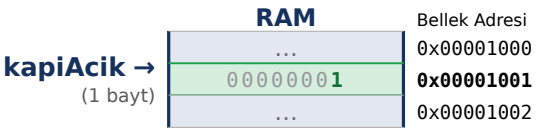
1 bayt = 8 bit
(her bit yalnızca 0 ya da 1)

1. Bu veri **ne tür** bir veri?
   (sayı mı, karakter mi, doğru/yanlış mı?)
2. RAM'de **kaç kutu (bayt)** ayrılmalı ve **hangi işlemler** yapılabilir?

bool — 1 bayt

- İki değer: **true** (1) ya da **false** (0)
- Karar yapılarında kullanılır (if, while)
- Tek bit yeter; yine de 1 tam bayt ayrılır

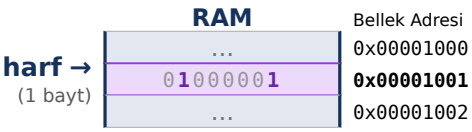
```
bool kapiAcik = true;
```



char — 1 bayt

- Tek karakter: harf, rakam karakteri, noktalama
- ASCII sayısıyla saklanır: 'A' = 65
- Matematiksel işlem yapılmaz: '5' ≠ 5

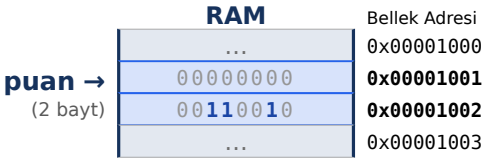
```
char harf = 'A';
```



int — 2 bayt

- 2^{16} = 65536 değer: −32768 ... +32767
- Yalnızca tam sayı: 7 / 2 = 3, 7 % 2 = 1
- Toplama, çıkarma, çarpma, bölme, mod

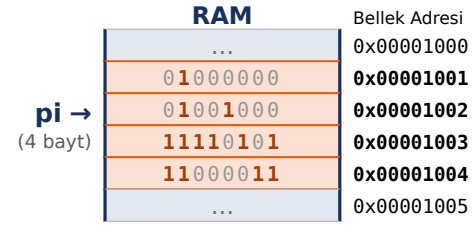
```
int puan = 50;
```



float — 4 bayt

- Virgüllü sayılar: 3.14 (kodda nokta ile)
- int'in 2 katı yer; 7 / 2 = 3.5 (küsuratlı)
- Çok geniş aralık, IEEE 754 biçimi

```
float pi = 3.14;
```



Karakterden Metne

Tek bir **char** 1 bayt yer kaplar. Birden fazla karakterden oluşan bir metin için her harfe RAM'de art arda (bitişik) 1 baytlık kutular ayrılır.

Toplam bellek alanı = karakter sayısı × 1 bayt

"Ada Lovelace" → 12 karakter (boşluk dahil) = **12 bayt**. Boşluk karakteri (SP, ASCII 32) de 1 bayt yer kaplar.

Neden önemli? Büyük veri türleri hem RAM'de daha fazla yer kaplar hem de işlemciyi daha fazla meşgul eder; bu yüzden gerekmedikçe **int** yerine **float** kullanılmamalıdır.

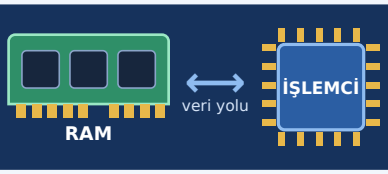
RAM
karakter · 8 bit · ASCII

			Bellek Adresi
A	01000001	65	0x00001000
d	01100100	100	0x00001001
a	01100001	97	0x00001002
SP	00100000	32	0x00001003
L	01001100	76	0x00001004
o	01101111	111	0x00001005
v	01110110	118	0x00001006
e	01100101	101	0x00001007
l	01101100	108	0x00001008
a	01100001	97	0x00001009
c	01100011	99	0x0000100A
e	01100101	101	0x0000100B

Not: Anlatımda C++ ve 16 bit sistemler (Arduino Uno, Turbo C) esas alınmıştır; böylece veri türlerinin donanımsal mantığı daha rahat anlaşılır. Derslerimizde Python kullanılacaktır; Python'da veri türleri ve bellek yapısı farklı işler.

RAM ve İşlemci İlişkisi

İşlemci, RAM'den aldığı veriyi işleyip tekrar RAM'e yazar.
Veri ne kadar büyükse bu yolculuk o kadar uzun sürer.



Büyük veri türü (daha çok bayt) → **RAM'de daha fazla yer** → **Veri yolunda daha çok bayt taşınır** → **İşlemci daha uzun süre meşgul olur**

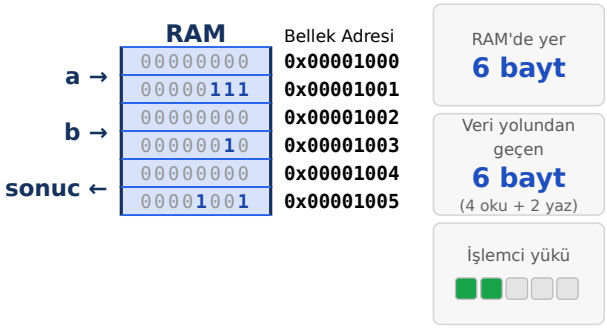
İşlemci RAM ile nasıl çalışır? (sonuc = a + b)



1. **Oku:** işlemci **a** ve **b** değerlerini RAM'den veri yolu ile alır.
2. **İşle:** toplama işlemini yapar.
3. **Yaz:** sonucu RAM'e geri yazar.

int — 2 bayt

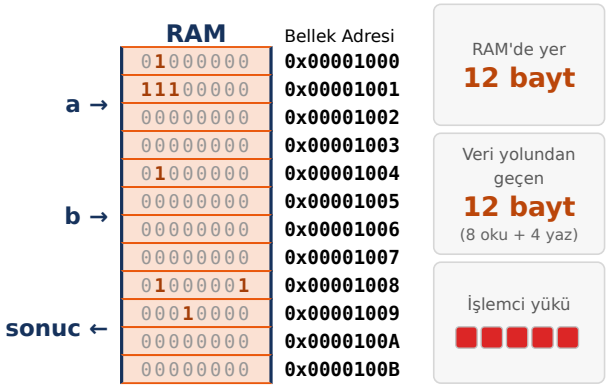
```
int sonuc = a + b;
```



a = 7, b = 2 için toplam sadece **6 bayt** RAM'e yazılıp okunur. Küçük bir işlemcide (ör. 8 bitlik Arduino Uno) bile 2 küçük adımda (düşük bayt, sonra yüksek bayt) toplanır.

float — 4 bayt

```
float sonuc = a + b;
```



Aynı işlem için **12 bayt** taşınır: int'in **2 katı**. FPU olmayan işlemcilerde (ör. Arduino Uno) float, yazılımla çok daha fazla adımda hesaplanır.

Metin (char dizisi): her karakter 1 bayt, boşluk da dahil

```
char isim[7] = {'A', 'l', 'i', ' ', 'C', 'a', 'n'};
```

- **"Ali Can"** = 6 harf + 1 boşluk = **7 karakter = 7 bayt**
- **Boşluk da bir karakterdir:** ASCII 32, RAM'de 1 bayt yer kaplar ve işlemci onu da okur.
- Metni kopyalarken, karşılaştırırken ya da ekrana yazarken işlemci **her karakter için bir adım** atar.

A	l	i	SP	C	a	n
adım 1	adım 2	adım 3	adım 4	adım 5	adım 6	adım 7

RAM

karakter · 8 bit · ASCII			Bellek Adresi
A	01000001	65	0x00001000
l	01101100	108	0x00001001
i	01101001	105	0x00001002
SP	00100000	32	0x00001003
C	01000011	67	0x00001004
a	01100001	97	0x00001005
n	01101110	110	0x00001006



1. **Tam sayı: int** — Küsurat gerekmedikçe float kullanma: daha çok RAM, daha çok işlemci süresi.
2. **Uygun türü seç** — Değerin aralığına yeten en küçük türü kullan.
3. **Metin uzadıkça** — Boşluklar dahil her karakter RAM'de yer tutar ve işlenir; yük artar.
4. **Eski donanım** — RAM ve işlemci sınırlıydı; veri türleri verimli bellek yönetimi için doğdu.

Not: Bayt sayıları C++ ve 16 bit sistemler (Arduino Uno, Turbo C) esas alınarak verilmiştir. Gerçek işlem süresi işlemciye göre değişir; ama RAM'de yer kaplayan her bayt, veri yolundan geçer ve işlemcinin işini artırır. Python'da bellek yapısı farklıdır.

12. Python Dili

Python; yüksek seviyeli, yorumlanan, nesne yönelimli ve geniş bir kütüphane ekosistemine sahip bir programlama dilidir. 1991 yılında Guido van Rossum tarafından geliştirilen bu dil, adını van Rossum'un beğeniyle takip ettiği İngiliz komedi grubu Monty Python'ın Monty Python's Flying Circus adlı televizyon programından almıştır.

Günümüzde bilgisayar donanımları, programlama dillerinin ilk ortaya çıktığı dönemlere kıyasla inanılmaz bir kapasiteye ulaşmıştır. Bu sayede Python gibi dillerin getirdiği çalışma zamanı maliyeti modern donanımlarla büyük ölçüde tolere edilmekte; çoğu uygulama için fazlasıyla yeterli ve pratik bir performans sunmaktadır.

İki Sayının Toplamını Alan Kod Örneği

C Dili

```c
#include <stdio.h>

int main() {
    int sayi1, sayi2, toplam;

    printf("Birinci sayi: ");
    scanf("%d", &sayi1);

    printf("Ikinci sayi: ");
    scanf("%d", &sayi2);

    toplam = sayi1 + sayi2;
    printf("Toplam: %d\n", toplam);

    return 0;
}
```

Python Dili

```python
sayi1 = int(input("Birinci sayı: "))
sayi2 = int(input("İkinci sayı: "))

toplam = sayi1 + sayi2
print(f"Toplam: {toplam}")
```

13. Tür Dönüşümü (Type Conversion / Type Casting)

Bilgisayarlar bellekteki verileri türlerine göre farklı formatlarda saklandığını gördük. Bir verinin türünü başka bir türe çevirme işlemine **tür dönüşümü** denir.

13.1. Kullanıcı Girişlerinin Sayıya Dönüştürülmesi (Metin $\rightarrow$ Sayı)

Klavyeden girilen veya bir metin dosyasından okunan tüm veriler, sisteme ilk olarak karakter dizisi (metin / string) biçiminde ulaşır.

- Kullanıcı ekrana 25 yazdığında bilgisayar bunu sayısal 25 olarak değil; yan yana duran **'2'**(ASCII: 50) ve **'5'**(ASCII: 53) karakterleri olarak algılar.
- Karakter tabanlı bir veriyle matematiksel işlem yapılamaz. Örneğin iki metinsel veri toplanmak istendiğinde matematiksel toplama ($25 + 10 = 35$) yerine metin birleştirme ("25" + "10" = "2510") gerçekleşir veya derleyici hata verir.
- Bu değerlerin matematiksel işlemlere girebilmesi için metin biçiminden uygun sayı türüne (int veya float) dönüştürülmesi (parse/cast edilmesi) şarttır.

Örnek Python Kodu Kullanıcının girdiği tam sayı değere 25 ekleyerek sonucu ekrana yazdıran kod.

```python
puan = int(input("Puan Giriniz: "))
print("Puanı:", puan)
puan = puan + 25
print("Güncellenmiş Puan:", puan)
```

Python Ekran Çıktısı

```
Puan Giriniz: 50
Puanı: 50
Güncellenmiş Puan: 75
```

```python
puan = int(input("Puan Giriniz: "))
```

Program, kullanıcıdan “Puan Giriniz:” iletisiyle bir tam sayı değeri beklemektedir. Python’da input() fonksiyonu klavyeden veri almak için kullanılır. Ancak daha önce belirttiğimiz gibi, input() fonksiyonu girilen veriyi ne olursa olsun her zaman bir metin (string) olarak okur. Bu metinsel değerin matematiksel hesaplamalarda veya karşılaştırmalarda kullanılabilmesi için int() fonksiyonu ile tam sayıya dönüştürülmesi şarttır.

```python
veri = input("Puan Giriniz: ")  # 1. Aşama: Metin olarak alındı ("50")
puan = int(veri)                # 2. Aşama: Tam sayıya dönüştürüldü (50)
```

Dileyenler bu işlemi tek satırda iç içe yapmak yerine, önce veriyi bir değişkene metin olarak alıp ardından ikinci bir satırda **int()** fonksiyonuyla sayıya dönüştürerek iki aşamada da gerçekleştirebilir. Fakat bu tek satır kod yazmaya göre daha uzundur.

```python
# Kullanıcı girişleri metin olarak alınır
birinci_sayi = input("Birinci Sayıyı Girin: ")
ikinci_sayi = input("İkinci Sayıyı Girin: ")
ucuncu_sayi = input("Üçüncü Sayıyı Girin: ")

# Girilen 3 değişken float türüne dönüştürülür
birinci_sayi = float(birinci_sayi)
ikinci_sayi = float(ikinci_sayi)
ucuncu_sayi = float(ucuncu_sayi)

bolum = (birinci_sayi + ikinci_sayi) / ucuncu_sayi
print("Bölüm:", bolum)
```

Örnek Python Kodu : Kullanıcı girişlerinin **metin türünden float** tipine dönüşümü yapılır.

Python Ekran Çıktısı

```
Birinci Sayıyı Girin: 1.2
İkinci Sayıyı Girin: 1.8
Üçüncü Sayıyı Girin: 2.5
Bölüm: 1.2
```

13.2. Otomatik (Örtük / Implicit) Dönüşüm:

Farklı türler aynı matematiksel işleme girdiğinde, veri kaybını önlemek için küçük tür otomatik olarak daha geniş ve kapsayıcı olan türe yükseltilir:

`int` + `float` → `float`

`int` - `float` → `float`

`int` * `float` → `float`

`int` / `float` → `float`

```python
adet = 5            # int türünde
birim_fiyat = 2.5   # float türünde

# Python arka planda 'adet' değerini 5.0 yapar
toplam_tutar = adet * birim_fiyat # 5.0 * 2.5 = 12.5(float)

print(toplam_tutar) # Çıktı: 12.5
```

13.3. Bilinçli (Açık / Explicit) Dönüşüm:

Programcının veri kaybını göze alarak veya işlemi yönlendirmek için türü zorla değiştirmesidir.

13.3.1. Hassasiyeti Korumak İçin Tam Sayıyı Kesrli Sayıya Çevirmek (int ⟶ float):

Daha hassas değerler gerektiğinde tam sayı değerleri kasten kesirli değerlere dönüştürülebilir.

```python
puan = 100
ondalikli_puan = float(puan)  # 100 -> 100.0

print(ondalikli_puan)  # Çıktı: 100.0
```

13.3.2. Ondalık Kısmı Atmak İçin (float ⟶ int):

Ondalıklı bir sayının virgülden sonraki kısmını silip yalnızca tam kısmını elde etmek gerektiğinde bu dönüşümden yararlanılır.

```
ortalama = 85.85
tam_not = int(ortalama)  # Virgülden sonrası silinir, tam_not = 85 olur

print(tam_not)  # Çıktı: 85
```

14. Değişken Tanımlama Kuralları

Yazılan kaynak kodların Python yorumlayıcısı tarafından doğru şekilde işlenebilmesi için değişken adları belirlenirken Python'un söz dizimi (**syntax**) kurallarına uyulması zorunludur.

- **Karakter Kısıtlaması:** Değişken isimleri sadece alfabetik karakterler (**A-Z, a-z**), rakamlar (**0-9**) ve alt çizgi (_) karakterini barındırabilir. Bunların dışındaki hiçbir özel sembol (nokta, virgül, soru işareti, tire, boşluk vb.) değişken isminde yer alamaz.
- **Başlangıç Kriteri:** Bir değişken ismi kesinlikle bir rakam ile başlayamaz. İsimler ya alfabetik bir karakterle ya da alt çizgi (_) işaretiyle başlamak zorundadır. Örneğin `3etap_skor` hatalı bir söz dizimi iken, **etap_skor_3** veya **_etap3** kurallara uygundur.
- **Büyük-Küçük Harf Duyarlılığı (Case-Sensitivity):** Python dili, büyük ve küçük harfleri farklı karakterler olarak değerlendirir. Bu nedenle aynı harflerden oluşsa bile yazımında büyük-küçük harf farkı bulunan kelimeler, bellekte tutulan verilere erişmek için kullanılan farklı değişken adları olarak kabul edilir. Örneğin **asistan_adi**, **Asistan_Adi** ve **ASISTAN_ADI** ifadeleri Python yorumlayıcısı tarafından birbirinden bağımsız üç farklı değişken adı olarak değerlendirilir.
- **Resmî Dil Anahtarları (Reserved Words):** Python dilinin kendi iç mekanizmasını yönetmek ve komutları işletmek adına kendine ayırdığı bazı özel kelimeler (keywords) değişken adı olarak kullanılamaz. Örneğin **if**, **else**, **True**, **False**, **while**, **for** gibi kelimeler Python için özel bir anlam ifade ettiğinden doğrudan değişken ismi yapılamaz.
- **Türkçe Karakter Kullanımı:** Kodlama dillerinin ve yazılım araçlarının küresel standardı gereği, değişken isimlendirirken **Türkçe'ye özgü harfleri kullanmaktan kesinlikle kaçınınız!**.

Türkçe Karakterlerin İngilizce Karşılıkları

Türkçe	Dönüşüm	İngilizce
ç / Ç	⟶	**c / C**
ğ / Ğ	⟶	**g / G**
ı / İ	⟶	**i / I**
ö / Ö	⟶	**o / O**
ş / Ş	⟶	**s / S**
ü / Ü	⟶	**u / U**

Değişken Kullanım Örnekleri

Hatalı Kullanım	Dönüşüm	Doğru Kullanım
`doğumTarihi`	⟶	**`dogumTarihi`**
`ürünFiyatı`	⟶	**`urunFiyati`**
`şubeKodu`	⟶	**`subeKodu`**
`kayıtDurumu`	⟶	**`kayitDurumu`**
`şifresi`	⟶	**`sifresi`**
`öğrenci_yaşı`	⟶	**`ogrenci_yasi`**
`ücret_miktarı`	⟶	**`ucret_miktari`**
`ilk_değer`	⟶	**`ilk_deger`**

15. Değişken Adlandırma Standartları (Notasyonlar)

15.1. Yılan Gösterimi (snake_case)

Tüm harflerin küçük yazıldığı ve kelimelerin birbirine alt çizgi (_) karakteriyle bağlandığı yöntemdir. Python literatüründe değişken ve fonksiyon isimleri için bu standardın kullanılması (örneğin **kullanici_giris_tarihi, asistan_bellek_kapasitesi** vb.) kurumsal olarak tavsiye edilir.

15.2. Camel Case (Deve Notasyonu) Kuralı

Değişken adları tanımlanırken kelimeler arasında boşluk bırakılamaz. Birden fazla kelimeden oluşan değişkenleri okunaklı kılmak için dünya çapında standart olan **camelCase** yöntemi kullanılır.
İlk kelimenin tüm harfleri küçük başlar. Sonraki her kelimenin ilk harfi büyük yazılır (kelimelerin ortasındaki hörgüç görünümü gibi).

Değişken Adlandırma Kuralları (Naming Conventions)

Yanlış / Zor Okunan Biçim	snake_case (Python)	camelCase
personelsayisi	personel_sayisi	personelSayisi
dogumyil	dogum_yil	dogumYil
ogrencinotortalamasi	ogrenci_not_ortalamasi	ogrenciNotOrtalamasi
toplamfiyat	toplam_fiyat	toplamFiyat
kullaniciadi	kullanici_adi	kullaniciAdi
maksimumhizsiniri	maksimum_hiz_siniri	maksimumHizSiniri
sonodemetarihi	son_odeme_tarihi	sonOdemeTarihi

16. Sabitler

Bir yazılım mimarisinde programın çalışma süresince değiştirilmemesi amaçlanan ve aynı değeri koruması beklenen verilere sabit (**constant**) adı verilir. Sabit olarak kullanılması amaçlanan isimler, tüm harfleri büyük olacak ve kelimeler alt çizgiyle (_) ayrılacak biçimde yazılır.

```
# Doğru Sabit Kullanım Standardı (BÜYÜK HARF)
PI_SAYISI = 3.14159
MAKSIMUM_DENEME_SAYISI = 3
SISTEM_PORT_NUMARASI = 443
VARSAYILAN_DIL = "TR"
KARSILAMA_MESAJI = " Sisteme Hoş Geldiniz!"
GELISTIRICI_MODU_ACIK = True
```

17. Açıklama Satırı (Comment line)

Yazılım projelerinde, derleyicinin görmezden geldiği ancak yazılımcıların kendileri için veya diğer yazılımcılar için bıraktığı notlara açıklama sataı (comment line) denir. Python'da iki tür açıklama satırı uygulaması vardır.

- **#** ile başlayan tek satırlık açıklama satırı
- **"""..."""** Çok satırlı açıklama satırları üç adet tırnak ile açılan ve yine üç adet tırnak ile kapanan alan içine yazılan açıklama ifadeleri ile yapılır.

Açıklamasız

```
a = 50
b = 20
c = a * b / 100
print(c)
```

Açıklamalı

```
# Ürünün fiyatı
a = 50
# İndirim oranı (%)
b = 20
# İndirim miktarı
c = a * b / 100
print(c)
```

Açıklama Satırının Kullanım Amaçları

```
# 1) Kodu açıklamak için
yas = 17  # öğrencinin yaşı

# 2) Kodu geçici olarak devre dışı bırakmak için
print("Merhaba")
# print("Bu satır çalışmaz")

# 3) Yapılacakları not etmek için
# TODO: Kullanıcıdan yaş bilgisi alınacak
```

Çok Satırlı Açıklama ile Program Başlığı

```
"""
Program : Daire Alanı Hesaplama
Yazar   : Ahmet Tuna POTUR
Tarih   : 01.11.2026
Açıklama: Yarıçapı verilen dairenin alanını hesaplar.
"""
pi = 3.14        # pi sayısı (yaklaşık değer)
r = 5            # yarıçap
alan = pi * r * r
print(alan)      # Ekrana 78.5 yazar
```

> **Dikkat:** Açıklama satırları programın çalışmasını etkilemez, ancak kodun okunabilirliğini artırır. Gereksiz ve her satır için tekrarlanan açıklamalardan kaçınılmalı; kodun **ne yaptığından çok neden yaptığı** anlatılmalıdır.

```
# Bu tam satırlık açıklama
x = 10  # satır içi (inline) açıklama
```

```
# Bu fonksiyon iki sayıyı toplar,
# ve geriye döndürür.
def topla(a, b):
    return a + b
# fonksiyon sonlandırıldı
```

```
"""
Bu teknik olarak bir string değişmezidir, ancak bir
değişkene atanmadığı için yorum bloğu gibi davranır.
"""
print("Kod çalışıyor")
```

18. Operatörler ve İşlem Önceliği

Operatör, bir veya birden fazla değer üzerinde **matematiksel hesaplama** yapan, değerleri **karşılaştıran** veya **mantıksal işlemler** yürüterek yeni bir sonuç üreten özel **sembol** ya da **anahtar kelimelerdir**. Operatörün üzerinde işlem yaptığı değerlere veya değişkenlere **operand** (işlenen) denir.

```
sonuc = 5 + 3
```

- **5** ve **3** → **operand** (işlenen değerler)
- **+** → **operatör** (işlemi yapan sembol)
- **=** → **atama operatörü** (sonucu `sonuc` değişkenine yerleştirir)

Python'daki operatörleri dört ana grupta inceleyeceğiz

Grup	Ne işe yarar?	Operatörler
Atama	Bir değişkene değer yerleştirir.	`= += -= *= /= //= %= **=`
Aritmetik	Matematiksel hesap yapar.	`+ - * / // % **`
Karşılaştırma	İki değeri kıyaslar; sonuç `True` veya `False` olur.	`== != > < >= <=`
Mantıksal	Koşulları birleştirir veya tersine çevirir.	`and or not`

18.1. Atama Operatörleri

Matematikte "eşittir" anlamına gelen **=** sembolü, programlama dillerinde atama operatörüdür; sağındaki değeri alır ve soldaki değişkene aktarır.

> **Atama Her Zaman Sağdan Sola Çalışır** Eğer bir hesaplama işlemi yapılıyorsa, önce operatörün sağ tarafı hesaplanır; ardından elde edilen sonuç soldaki değişkene atanır.

18.1.1. Atama Operatörü (=): Değişken Oluşur veya Güncellenir

Bir değişkene ilk kez değer atadığımızda, atanan değer için bellek (RAM) üzerinde bir yer ayrılır ve değer bu alana yazılır. Değişken adı ise bu bellek alanını gösteren bir **etikettir (işaretçidir)**.

```
yas = 15          # yas değişkeni oluştu, bellekte 15 değeri saklandı
isim = "Ayşe"     # isim değişkeni oluştu, bellekte "Ayşe" saklandı
pi_sayisi = 3.14  # pi_sayisi değişkeni oluştu
```

Var olan bir değişkene yeniden atama yaptığımızda yeni bir değişken oluşmaz; değişkenin bulunduğu bellek alanında ki eski değerin üzerine yeni değer yazılır ve eski değer kaybolur.

Örnek Python Kodu İki kez atama yapma.

```
puan = 50
print("Puan:", puan)
puan = 75
print("Güncel puan:", puan)
```

Python Ekran Çıktısı

```
Puan: 50
Güncel puan: 75
```

Güncelleme yaparken sağ tarafta değişkenin önceki değeri ile hesaplama yapılır, sonra sonuç değişkene atanır.

```
puan = 50
puan = puan + 25 # 50 + 25 = 75 hesaplanır, puan'a atanır
print(sayac)  # 75
```

Python'da tür bilgisi değere bağlı olduğu için, değişken yeni atanan değerin türünü alır:

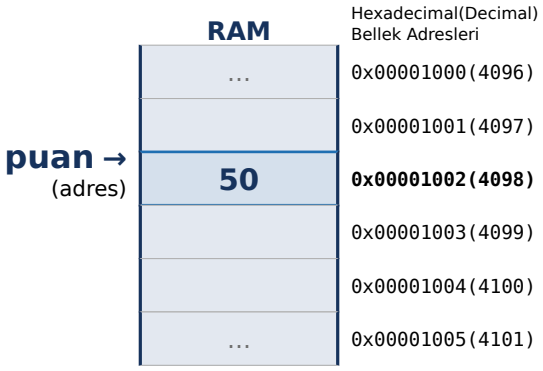
```
puan = 50     # int
puan = 55.43  # artık float
puan = "elli" # artık str
```

Bellekte Değerin Güncellenmesi

1 Değer Atama

Giriş yapılan **50** değeri, RAM'de **puan** ismiyle bir adrese kaydedilir.

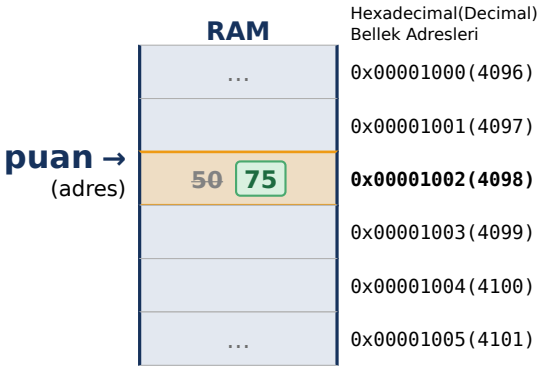
```
puan = 50
```



2 Değer Güncelleme

Giriş yapılan **75** değeri, RAM'de bulunan **puan** değişkeninin önceki değerinin üzerine yazılır.

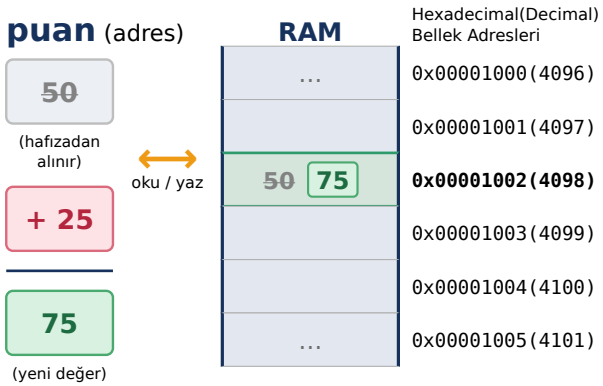
```
puan = 75
```



3 Mevcut ile Güncelleme

Güncellenen değişkenin eski değeri kullanılarak yeni değer yazılır.

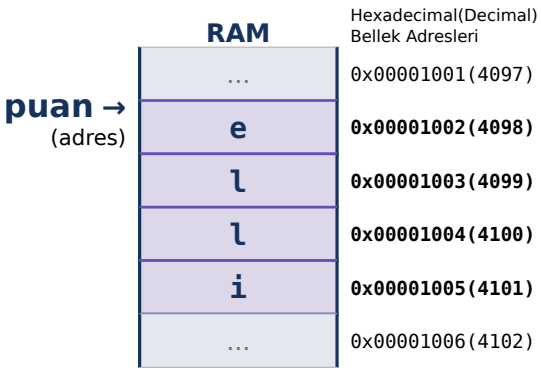
```
puan = puan + 25
```



4 Değişken Türü ile Güncelleme

RAM'deki sayısal alan silinir, yerine metin dizisi oluşturulur.

```
puan = "elli"
```



18.1.2. Bileşik Atama Operatörleri

sayac = sayac + 1 gibi **"değişkeni kendi değeriyle işleme sokup tekrar kendisine yazma"** işlemi o kadar sık yapılır ki bunun için kısaltmalar vardır:

Bileşik Atama Operatörleri

Operatör	Kısa Yazım	Uzun Karşılığı	`a = 10` iken sonuç
+=	`a += 3`	`a = a + 3`	13
-=	`a -= 3`	`a = a - 3`	7
*=	`a *= 3`	`a = a * 3`	30
/=	`a /= 4`	`a = a / 4`	2.5
//=	`a //= 4`	`a = a // 4`	2
%=	`a %= 4`	`a = a % 4`	2
**=	`a **= 2`	`a = a ** 2`	100

```
1 bakiye = 100
2 bakiye += 50
3 bakiye -= 30
4 bakiye *= 2
5 print("Bakiye:", bakiye)
6
7 mesaj = "Merhaba"
8 mesaj += " Dünya"
9 print(mesaj)
```

Örnek Python Kodu Bakiye üzerinde bileşik atama operatörleri kullanılır.

Python Ekran Çıktısı

```
Bakiye: 240
Merhaba Dünya
```

18.2. Aritmetik Operatörler

Python Aritmetik Operatörleri: Matematiksel İşlemleri Yapar

Operatör	Adı	Örnek	Sonuç	Açıklama
+	Toplama	`10 + 3`	13	İki sayıyı toplar.
-	Çıkarma	`10 - 3`	7	Soldakinden sağdakini çıkarır.
*	Çarpma	`10 * 3`	30	İki sayıyı çarpar.

Operatör	Adı	Örnek	Sonuç	Açıklama
`/`	Bölme	`10 / 4`	`2.5`	Sonuç her zaman `float` olur.
`//`	Tam bölme	`10 // 4`	`2`	Bölümün tam sayı kısmını verir (küsuratı atar).
`%`	Mod (kalan)	`10 % 4`	`2`	Bölmeden kalanı verir.
`**`	Üs alma	`2 ** 3`	`8`	Soldaki sayının sağdaki kuvvetini alır.

Örnek Python Kodu İki sayı üzerinde aritmetik operatörlerin sonuçları.

```python
a = 17
b = 5

print("Toplam:", a + b)
print("Fark:", a - b)
print("Çarpım:", a * b)
print("Bölüm:", a / b)
print("Tam bölüm:", a // b)
print("Kalan:", a % b)
```

Python Ekran Çıktısı

```
Toplam: 22
Fark: 12
Çarpım: 85
Bölüm: 3.4
Tam bölüm: 3
Kalan: 2
```

Not: Tam bölünen sayılarda bile sonuç `float` türüne dönüşür

```python
print(10 / 4)          # 2.5
print(10 / 5)          # 2.0   (2 değil!)
```

Tam Bölme (`//`) ve Mod (`%`): Bir bölme işlemini ilkokuldaki gibi düşünün: **bölünen = bölen × bölüm + kalan**. `//` operatörü bölümü, `%` operatörü kalanı verir.

```python
print(17 // 5)   # 3   (17 içinde 3 tane 5 var)
print(17 % 5)    # 2   (3 × 5 = 15, geriye 2 kalır)
```

Tam Bölme ve Mod Örnekleri

Bölünen ve Bölen	`//` (bölüm)	`%` (kalan)	Kontrol
17 ve 5	3	2	`3 * 5 + 2 = 17`
20 ve 4	5	0	`5 * 4 + 0 = 20`
100 ve 30	3	10	`3 * 30 + 10 = 100`

Mod operatörünün en sık kullanıldığı iki yer:

```python
# 1) Tek mi çift mi?
sayi = 7
print(sayi % 2)    # 1  -> kalan 1 ise sayı tektir
print(10 % 2)      # 0  -> kalan 0 ise sayı çifttir

# 2) Bir sayı diğerine tam bölünüyor mu?
print(15 % 5)      # 0  -> 15, 5'e tam bölünür
print(15 % 4)      # 3  -> tam bölünmez
```

Örnek Python Kodu Dakikayı saat ve dakikaya çevirir.

```python
toplam_dakika = 135
saat = toplam_dakika // 60
dakika = toplam_dakika % 60
print(saat, "saat", dakika, "dakika")
```

Python Ekran Çıktısı

```
2 saat 15 dakika
```

Üs Alma (**)

```
print(2 ** 3)     # 8       (2 × 2 × 2)
print(5 ** 2)     # 25
print(9 ** 0.5)   # 3.0     (karekök almak: 0.5 kuvveti)
```

Metinlerle `+` ve `*`:
+ operatörü metinleri birleştirir, ***** operatörü bir metni belirtilen sayı kadar tekrarlar

Örnek Python Kodu Dakikayı saat ve dakikaya çevirir.

```
ad = "Ali"
soyad = "Yılmaz"
print(ad + " " + soyad)
print("Ha" * 3)
print("-" * 20)
print("Yaş: " + str(15))
```

Python Ekran Çıktısı

```
Ali Yılmaz
HaHaHa
--------------------
Yaş: 15
```

18.3. Aritmetik Operatörlerde İşlem Önceliği

Bir ifadede birden fazla operatör varsa Python işlemleri işlem önceliğine göre matematikteki sırayla yapar. Önceliği yüksek olan işlem önce hesaplanır.

Aritmetik Operatörlerin İşlem Önceliği

Öncelik	Operatör	Açıklama
1 (en yüksek)	`( )`	Parantez içi her zaman önce hesaplanır.
2	`**`	Üs alma
3	`-x`	Tek operandlı eksi (negatif işareti)
4	`* / // %`	Çarpma, bölme, tam bölme, mod (kendi aralarında eşit öncelikli)
5 (en düşük)	`+ -`	Toplama, çıkarma (kendi aralarında eşit öncelikli)

Eşit öncelikli operatörler soldan sağa işlenir:

```
print(10 - 4 - 3)      # 3      ((10 - 4) - 3)
print(100 / 10 * 5)    # 50.0   ((100 / 10) * 5)
```

```
print(2 + 3 * 4)         # 14
# 1) 3 * 4 = 12
# 2) 2 + 12 = 14
```

```
print((2 + 3) * 4)       # 20
# 1) (2 + 3) = 5
# 2) 5 * 4 = 20
```

```
print(2 + 3 ** 2 * 2)    # 20
# 1) 3 ** 2 = 9
# 2) 9 * 2 = 18
# 3) 2 + 18 = 20
```

```
print(7 + 9 / 3 * 2)    # 13.0
# 1) 9 / 3 = 3.0
# 2) 3.0 * 2 = 6.0
# 3) 7 + 6.0 = 13.0
```

İstisna: Üs alma (**), eşit öncelikli işlemlerin aksine **sağdan sola** çalışır.
Eksi işareti de ** operatöründen daha düşük önceliklidir:

```
print(2 ** 3 ** 2)     # 512  -> 2 ** (3 ** 2) = 2 ** 9
print((2 ** 3) ** 2)   # 64
print(-2 ** 2)         # -4   -> -(2 ** 2)
print((-2) ** 2)       # 4
```

Pratik öneri: Sıradan emin değilseniz **parantez kullanın**. Parantez hem hatayı önler hem de kodu okuyan kişiye niyetinizi gösterir.

```
n1 = 80
n2 = 90
n3 = 70
ortalama = (n1 + n2 + n3) / 3
print("Ortalama:", ortalama)

c = 25
f = c * 9 / 5 + 32
print("Fahrenheit:", f)
```

Örnek Python Kodu Not ortalaması ve sıcaklık dönüşümü.

Python Ekran Çıktısı

```
Ortalama: 80.0
Fahrenheit: 77.0
```

```
# Dikdörtgenin alanı ve çevresi
en = 6
boy = 4
alan = en * boy         # 24
cevre = 2 * (en + boy)  # 20
```

```
# Bir sayının son basamağı ve
# son basamağı atılmış hali
sayi = 4827
print(sayi % 10)    # 7
print(sayi // 10)   # 482
```

18.4. Karşılaştırma Operatörleri

Karşılaştırma operatörleri iki değeri kıyaslar ve sonuç olarak her zaman `True` (doğru) veya `False` (yanlış) üretir. Bu iki değer `bool` (mantıksal) veri türündedir.

Python Karşılaştırma Operatörleri

Operatör	Anlamı	Örnek	Sonuç	Açıklama
==	Eşittir	`5 == 5`	`True`	İki değer eşitse `True` olur.
!=	Eşit değildir	`5 != 3`	`True`	İki değer farklıysa `True` olur.
>	Büyüktür	`7 > 3`	`True`	Soldaki, sağdakinden büyükse `True` olur.
<	Küçüktür	`7 < 3`	`False`	Soldaki, sağdakinden küçükse `True` olur.
>=	Büyük eşittir	`5 >= 5`	`True`	Soldaki büyük **veya** eşitse `True` olur.
<=	Küçük eşittir	`4 <= 3`	`False`	Soldaki küçük **veya** eşitse `True` olur.

Örnek Python Kodu Bir sayının çeşitli karşılaştırma sonuçları.

```
sayi = 15
print(sayi > 10)
print(sayi == 15)
print(sayi != 15)
print(sayi <= 14)
print(sayi % 2 == 0)
```

Python Ekran Çıktısı

```
True
True
False
False
False
```

Not: Karşılaştırmanın sonucu da bir değerdir; bir değişkende saklanabilir:

```
yas = 16
resit_mi = yas >= 18
print(resit_mi)   # False
```

Dikkat: = ve == aynı şey değildir.

```
x = 5 # ATAMA: x'e 5 değerini yerleştir
x == 5  # KARŞILAŞTIRMA: x, 5'e eşit mi? -> True
```

Metinleri Karşılaştırma Metinler de karşılaştırılabilir. == operatörü büyük/küçük harfe duyarlıdır:

```
print("elma" == "elma")    # True
print("elma" == "Elma")    # False  (büyük/küçük harf farklı)
print("elma" != "armut")   # True
print("a" < "b")           # True   (alfabetik sıra / karakter kodu)
```

Farklı Türlerin Karşılaştırılması

```
print(3 == 3.0)        # True   (int ile float değer olarak eşit)
print("5" == 5)        # False  (metin ile sayı aynı şey değildir)
print(int("5") == 5)   # True   (türü dönüştürünce eşit)
```

Ondalıklı sayılarda == kullanırken dikkat: Bilgisayar ondalıklı sayıları tam kesinlikle saklayamaz. Bu yüzden `float` değerlerin == ile karşılaştırılması güvenilir değildir.

```
print(0.1 + 0.2)           # 0.30000000000000004
print(0.1 + 0.2 == 0.3)    # False
```

18.5. Mantıksal Operatörler

Mantıksal operatörler, `True` / `False` değerlerini ve karşılaştırma sonuçlarını birleştirmek veya tersine çevirmek için kullanılır. Üç tanedir: `and` (ve), `or` (veya), `not` (değil).

Python Mantıksal Operatörleri

Operatör	Anlamı	Ne zaman True olur?	Günlük Hayattan Örnek
and	Ve	İki koşul da `True` ise.	"Hava güzel **ve** ödevim bitti ise parka giderim."
or	Veya	En az bir koşul `True` ise.	"Param **veya** kartım varsa alışveriş yapabilirim."
not	Değil	Koşul `False` ise (tersine çevirir).	"Yağmur yağmıyorsa dışarı çıkarım."

Doğruluk Tabloları

and: Hepsi doğru olmalı

A	B	A and B
True	True	**True**
True	False	False
False	True	False
False	False	False

or: Biri doğru olsa yeter

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	**False**

not:Tersine çevirir

A	not A
True	False
False	True

Örnek Kod

```
print(True and True)  #True
print(True and False)
#False
print(False or True) #True
print(False or False)
#False
print(not True) #False
print(not False) #True
```

Karşılaştırma operatörleriyle birlikte kullanım

Yaş ve ehliyet durumuna göre koşul kontrolü.

```
yas = 20
ehliyet = True

print(yas >= 18 and ehliyet)
print(yas < 18 or yas > 60)
print(not ehliyet)
```

Python Ekran Çıktısı

```
True
False
False
```

```
sayi = 15
print(sayi % 3 == 0 and sayi % 5 == 0)   # True  (hem 3'e hem 5'e bölünüyor)
print(sayi % 2 == 0 or sayi > 10)        # True  (çift değil ama 10'dan büyük)
print(not (sayi > 20))                   # True
```

Örnek: Adım adım mantıksal ifade

```
not_degeri = 65
devam = 20

gecti = not_degeri >= 50 and devam <= 30
# 1) not_degeri >= 50  ->  65 >= 50  ->  True
# 2) devam <= 30       ->  20 <= 30  ->  True
# 3) True and True     ->  True
print(gecti)   # True
```

Kısa Devre (Short-Circuit): Python mantıksal ifadeyi **gereksiz yere sonuna kadar hesaplamaz**:

- **`and`** ifadesinde soldaki `False` ise sonuç kesindir (`False`); sağ taraf hiç değerlendirilmez.
- **`or`** ifadesinde soldaki `True` ise sonuç kesindir (`True`); sağ taraf hiç değerlendirilmez.

Bu özellik, hataya yol açabilecek bir işlemi önce kontrol etmek için çok kullanışlıdır.

```
x = 0
print(x != 0 and 10 / x > 2)   # False
# x != 0 zaten False -> 10 / x hiç çalışmaz, sıfıra bölme hatası oluşmaz

print(x == 0 or 10 / x > 2)    # True
# x == 0 zaten True -> sağ taraf hiç çalışmaz
```

18.6. Operatörler Arası İşlem Önceliği

Bir ifadede aritmetik, karşılaştırma ve mantıksal operatörler birlikte bulunabilir. Python bunları şu sırayla işler: **önce aritmetik, sonra karşılaştırma, en son mantıksal**.

Operatörler Arası İşlem Önceliği

Sıra	Grup	Operatörler	Not
1 (en yüksek)	Parantez	`( )`	Her zaman en başta işlenir.
2	Aritmetik	`**`	Sağdan sola işlenir.
3	Aritmetik	`-x`	Tek operandlı eksi.
4	Aritmetik	`* / // %`	Soldan sağa işlenir.
5	Aritmetik	`+ -`	Soldan sağa işlenir.
6	Karşılaştırma	`== != > < >= <=`	Hepsi eşit öncelikli.

Sıra	Grup	Operatörler	Not
7	Mantıksal	`not`	Mantıksal operatörler içinde en yüksek öncelikli.
8	Mantıksal	`and`	`not` operatöründen sonra işlenir.
9 (en düşük)	Mantıksal	`or`	En son işlenir.
En sonda	Atama	`= += -= ...`	Sağ taraf tamamen hesaplanır, sonra atanır.

Gerçek hayat, kampa katılım kuralı

```
yas = 15
veli_izni = True
saglik_raporu = False

katilabilir = yas >= 14 and yas <= 17 and (veli_izni or saglik_raporu)
# 1) yas >= 14                      ->  True
# 2) yas <= 17                      ->  True
# 3) (veli_izni or saglik_raporu)   ->  True or False  ->  True
# 4) True and True and True         ->  True
print(katilabilir)  #True
```

Özet:

- Operatör, değerler üzerinde işlem yapan semboldür.
- **=** bir **atama** operatörüdür: ilk atamada değişken oluşur, sonraki atamalarda değeri güncellenir.
- Aritmetik operatörler sayısal sonuç, karşılaştırma ve mantıksal operatörler `bool` sonuç üretir.
- Öncelik sırası: **parantez** → **aritmetik** → **karşılaştırma** → **`not`** → **`and`** → **`or`**.
- Emin olmadığınız yerde parantez kullanın.

19. Fonksiyonların Kullanımı

Kod tekrarı; bir görev için aynı kodun gerekli olduğu her yerde yeniden yazılmasıdır. Kod tekrarı yapıldığında derleyici veya yorumlayıcı hata vermez; fakat hataların düzeltilmesinin zorlaşması ve kodun okunaksız hâle gelmesi gibi birçok dezavantajı vardır. **Kod tekrarından kesinlikle kaçınılmalıdır!** Yazılım dünyasında bu ilkeyi ifade etmek için sıkça "Don't Repeat Yourself" (**DRY** - Kendini Tekrar Etme) prensibi kullanılır.

Fonksiyonlar kod tekrarını önleyen en temel araçlardandır. Belirli bir görevi yerine getiren kod blokları bir fonksiyon çatısı altında toplanır ve ihtiyaç duyuldukça yalnızca fonksiyonun adı çağrılır. Fonksiyonlar, kendilerine gönderilen argümanları (girdi değerlerini) işleyerek ekrana bir çıktı verebilir veya hesaplanan sonucu geriye bir değer olarak döndürebilir.

Aşağıda, kesirli bir sayının virgülden sonraki iki basamağını elde etmek için kullanılan manuel bir mantık gösterilmiştir. Bu işlem için her defasında 5 satır kod yazdığımızı varsayalım. Programın farklı yerlerinde bu işlemi yalnızca üç kez tekrar etmemiz bile 15 satırlık gereksiz bir kod kalabalığı yaratacaktır. Oysa bunun yerine Python'ın hazır `round()` fonksiyonunu kullandığımızda aynı işlem tek bir satıra iner.

Fonksiyonları Kullanmanın Yarattığı Fark

```
sayi = 3.14159
sayi2 = sayi*100
sayi3 = sayi2//1
sayi4 = sayi3/100
print(sayi4)  #3.14
```

↓

```
round(3.14159, 2) #3.14
```

```
sayi = 5.03123
sayi2 = sayi*100
sayi3 = sayi2//1
sayi4 = sayi3/100
print(sayi4)  #5.03
```

↓

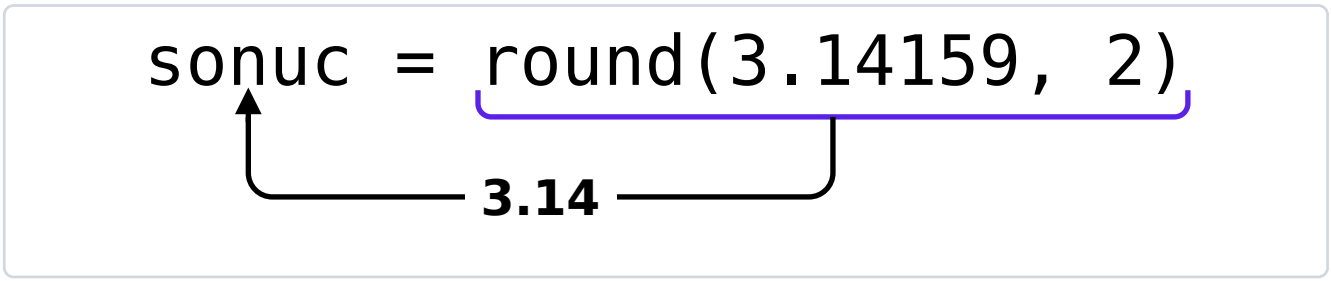
```
round(5.03123, 2) #5.03
```

```
sayi = 7.21013
sayi2 = sayi*100
sayi3 = sayi2//1
sayi4 = sayi3/100
print(sayi4)  #7.21
```

↓

```
round(7.21013, 2) #7.21
```

`round()` Fonksiyonunun Kullanımı



Yukarıdaki örnekte **`round()`** fonksiyonunun kullanımı gösterilmektedir. Bir fonksiyonu çalıştırmak (**çağırmak**) için fonksiyonun adından hemen sonra parantez açılıp kapatılır: **`round()`**.

`round()` fonksiyonu dışarıdan girdi kabul eden bir fonksiyondur. Fonksiyon çağrılırken parantez içerisine yazılan bu değerlere **argüman** denir. Burada girilen:

- **Birinci değer (`3.14159`):** İşlem yapılacak olan ondalıklı sayıyı,
- **İkinci değer (`2`):** Bu sayının virgülden sonra kaç basamağa yuvarlanacağını belirtir.

Fonksiyonun kendi içinde hesapladığı sonucu çağrıldığı yere iletmesine **değer döndürme (return)** denir. Yukarıdaki örnekte **`round()`** fonksiyonunun döndürdüğü sonuç, **`=`** atama operatörü kullanılarak **`sonuc`** isimli değişkene aktarılmıştır.

Fonksiyon Kavramları

Kavram	Açıklama	Örnek
Fonksiyon çağırma	Fonksiyonun adını yazıp yanına parantez koyarak çalıştırmak.	`print()`
Argüman	Fonksiyonun parantezi içine verilen değer. Birden fazlaysa virgülle ayrılır.	`round(3.14159, 2)`
Dönüş değeri	Fonksiyonun işini bitirince geri verdiği sonuç. Bir değişkene atanabilir.	`sonuc = round(3.14159, 2)`

```
1  sonuc = round(3.14159, 2)   # round fonksiyonu çağrıldı; 3.14159 ve 2 birer argüman
2  print(sonuc)                # 3.14
```

> **Dikkat:** Parantez olmadan yazılan `print` gibi bir fonksiyon ifadesi fonksiyonu çalıştırmaz. Fonksiyonun **çalışması** için `print()` biçiminde parantezle çağrılması gerekir.

19.1. Yerleşik (Built-in) Fonksiyonlar

Python'la birlikte hazır gelen fonksiyonlara **yerleşik fonksiyon** denir. Bu konuda göreceğimiz `print()` ve `input()` ile tür dönüşümünde kullandığımız `int()`, `float()` ve `str()` yerleşik fonksiyonlardır.

Sık Kullanılan Yerleşik Fonksiyonlar

Fonksiyon	Ne yapar?	Örnek	Sonuç
`print()`	Değerleri ekrana yazdırır.	`print("Merhaba")`	Ekrana `Merhaba` yazar
`input()`	Kullanıcıdan klavyeyle veri alır.	`input("Adınız: ")`	Yazılan metin
`int()`	Değeri tam sayıya çevirir.	`int("25")`	`25`
`float()`	Değeri ondalıklı sayıya çevirir.	`float("2.5")`	`2.5`
`str()`	Değeri metne çevirir.	`str(15)`	`"15"`
`type()`	Değerin türünü verir.	`type(5)`	`<class 'int'>`
`len()`	Metnin karakter sayısını verir.	`len("Python")`	`6`

Fonksiyon	Ne yapar?	Örnek	Sonuç
`round()`	Sayıyı yuvarlar.	`round(3.14159, 2)`	`3.14`
`abs()`	Mutlak değer verir.	`abs(-7)`	`7`
`max()` / `min()`	En büyük / en küçük değeri verir.	`max(3, 9, 5)`	`9`

Örnek Python Kodu Birkaç yerleşik fonksiyonun sonuçları.

```
print(len("Python"))
print(max(3, 9, 5))
print(min(3, 9, 5))
print(abs(-7))
print(round(3.14159, 2))
print(type(5))
print(type("5"))
```

Python Ekran Çıktısı

```
6
9
3
7
3.14
<class 'int'>
<class 'str'>
```

19.1.1. Dönüş Değeri Olan ve Olmayan Fonksiyonlar

`round()` gibi fonksiyonlar bir sonuç **geri verir** ve bu sonucu bir değişkene atayabiliriz. `print()` ise sonuç üretmez, işi ekrana yazmaktır. Bu yüzden `print()` çağrısının sonucunu bir değişkene atarsak `None` (yani "değer yok") elde ederiz.

```
a = round(2.678, 1)
print(a)          # 2.7

b = print("Merhaba")   # ekrana Merhaba yazılır
print(b)               # None
```

> **Ders notu:** Bir fonksiyonun ekrana yazdığı şey ile geri verdiği değer farklı kavramlardır. Ekrandaki yazıyı değişkende saklayamayız; ancak dönüş değerini saklayabiliriz.

20. input() ve print() Fonksiyonları

Çoğu program şu üç adımı izler: **veriyi al** (girdi), **işle**, **sonucu göster** (çıktı). `input()` girdi adımını, `print()` çıktı adımını yapar.

20.1. print(): Ekrana Yazdırma

`print()` fonksiyonu, parantez içine verilen değerleri ekrana yazar ve ardından bir alt satıra geçer. Metinler tırnak içinde yazılır; sayılar ve değişkenler tırnaksız yazılır.

Örnek Python Kodu Metin, sayı ve değişken yazdırma.

```
print("Merhaba Dünya")
print(2026)
print(7 + 3)

isim = "Ayşe"
print(isim)
print("isim")
```

Python Ekran Çıktısı

```
Merhaba Dünya
2026
10
Ayşe
isim
```

> **Dikkat:** `print(isim)` değişkenin içindeki değeri (`Ayşe`) yazdırır. `print("isim")` ise tırnak içindeki metni (`isim`) olduğu gibi yazdırır.

Boş `print()` çağrısı ekrana hiçbir şey yazmaz, yalnızca bir satır boşluk bırakır:

```
print("Birinci satır")
print()
print("Üçüncü satır")
```

20.2. input(): Kullanıcıdan Veri Alma

`input()` fonksiyonu program çalışırken klavyeden veri almamızı sağlar. Arka planda üç aşama gerçekleşir:

1. Python `input()` satırına gelince programın akışı **durur**.
2. Kullanıcı klavyeden bir şey yazıp **Enter** tuşuna basana kadar program bekler.
3. Enter'a basılınca yazılan her şey bir **metin (str)** olarak geri verilir ve sol taraftaki değişkene atanır. Program bir alt satırdan devam eder.

```
ad = input("Adınız nedir? ")
print("Merhaba", ad)
```

Parantez içine yazdığımız metin (`"Adınız nedir? "`) kullanıcıya gösterilen **yönlendirme mesajıdır**. Sonuna bir boşluk bırakmak, kullanıcının yazdığı cevabın mesaja yapışmamasını sağlar.

Örnek Python Kodu Kullanıcının adını alıp selamlama.

```
ad = input("Adınız nedir? ")
print("Merhaba", ad)
print("Tanıştığımıza memnun oldum!")
```

Python Ekran Çıktısı

```
Adınız nedir? Ayşe
Merhaba Ayşe
Tanıştığımıza memnun oldum!
```

20.2.1. input() Her Zaman Metin Verir

Kullanıcı `15` yazsa bile `input()` bunu sayı olarak değil, `"15"` **metni** olarak verir. Bu, tür dönüşümü konusunda gördüğümüz önemli bir kuraldır:

```
yas = input("Yaşınız: ")
print(type(yas))
print(yas + yas)
```

Python Ekran Çıktısı

```
Yaşınız: 15
<class 'str'>
1515
```

Metinlerde `+` toplama değil **birleştirme** yaptığı için `"15" + "15"` sonucu `1515` olur. Sayısal işlem yapacaksak `int()` veya `float()` ile dönüştürmemiz gerekir:

Örnek Python Kodu Doğum yılından yaş hesaplama.

```
dogum_yili = int(input("Doğum yılınız: "))
yas = 2026 - dogum_yili
print("Yaşınız:", yas)
```

Python Ekran Çıktısı

```
Doğum yılınız: 2010
Yaşınız: 16
```

Girilen Veriye Göre Dönüştürme

Beklenen veri	Kullanılacak yazım	Örnek
Metin (ad, şehir, mesaj)	`input("...")`	`sehir = input("Şehir: ")`
Tam sayı (yaş, adet)	`int(input("..."))`	`adet = int(input("Adet: "))`
Ondalıklı sayı (not, fiyat, boy)	`float(input("..."))`	`boy = float(input("Boy (m): "))`

> **Dikkat:** Kullanıcı `abc` gibi sayıya çevrilemeyecek bir şey yazarsa `int()` veya `float()` hata verir (`ValueError`) ve program durur. Hata yönetimini ileride göreceğiz; şimdilik kullanıcıdan doğru türde veri girildiğini varsayıyoruz.

20.3. print() ile Metin ve Değişkenleri Birlikte Yazdırma

Gerçek programlarda çoğu zaman sabit metin ile değişken değerlerini aynı satırda göstermek isteriz. Örneğin `Merhaba Ayşe, 16 yaşındasın.` Bunu yapmanın üç yolu vardır.

20.3.1. Virgül (,) ile Yazdırma

`print()` içindeki argümanlar virgülle ayrılır. Python aralarına **otomatik bir boşluk** koyar ve her değeri kendi türüne dokunmadan yazar. Bu yüzden sayı ile metni karıştırmak sorun çıkarmaz.

```
ad = "Ayşe"
yas = 16
print("Merhaba", ad, "yaşın", yas)   # Merhaba Ayşe yaşın 16
```

20.3.2. f-string (Formatlı Metin)

Modern ve en okunaklı yöntemdir. Metnin tırnağının hemen önüne **`f`** harfi yazılır; metnin içinde değişkenler **süslü parantez `{ }`** arasına konur. Python çalışırken süslü parantezin içindeki değişkeni değeriyle değiştirir.

Örnek Python Kodu Aynı çıktının f-string ile yazılışı.

```
ad = "Ayşe"
yas = 16
print(f"Merhaba {ad}, {yas} yaşındasın.")
```

Python Ekran Çıktısı

```
Merhaba Ayşe, 16 yaşındasın.
```

f-string'in avantajları:
- Boşlukları ve tırnakları tek tek düşünmeyiz; cümle nasıl görünecekse öyle yazarız.
- Kod, okunduğunda yazdığımız cümleye çok benzer.

```
# Aynı iş, üç farklı yazım
ad = "Ayşe"
yas = 16

print("Merhaba", ad + ",", yas, "yaşındasın.")        # virgül ve + karışık
print(f"Merhaba {ad}, {yas} yaşındasın.")             # f-string
```

20.3.3. f-string İçinde İşlem Yapmak

Süslü parantezin içine yalnızca değişken değil, **her türlü ifade** yazabiliriz.

Süslü parantez içinde ifade kullanımı:

```python
a = 8
b = 5
ad = "ayşe"

print(f"{a} + {b} = {a + b}")
print(f"{a} * {b} = {a * b}")
print(f"Ad: {ad.upper()}")
print(f"Ad {len(ad)} harflidir.")
```

Python Ekran Çıktısı

```
8 + 5 = 13
8 * 5 = 40
8 sayısı çift mi? True
Ad: AYŞE
Ad 4 harflidir.
```

20.3.4. f-string ile Sayıları Biçimlendirme

Süslü parantezin içinde değişkenin yanına : (iki nokta) koyup nasıl gösterileceğini belirtebiliriz. En çok kullanılanlar:

f-string Biçimlendirme Kalıpları

Kalıp	Örnek	Sonuç	Açıklama
`:.2f`	`f"{3.14159:.2f}"`	`3.14`	Virgülden sonra 2 basamak göster (yuvarlar).
`:.0f`	`f"{2.5678:.0f}"`	`3`	Virgülden sonra basamak gösterme.
`:,`	`f"{1234567:,}"`	`1,234,567`	Binlik ayırıcı koy.
`:,.2f`	`f"{1234567.891:,.2f}"`	`1,234,567.89`	Binlik ayırıcı ve 2 basamak.
`:.1%`	`f"{0.256:.1%}"`	`25.6%`	Yüzde olarak göster.
`:03d`	`f"{7:03d}"`	`007`	Toplam 3 basamak olacak şekilde başa sıfır ekle.

Örnek Python Kodu Fiyat hesabında biçimlendirme.

```python
fiyat = 249.9
adet = 3
toplam = fiyat * adet

print(f"Toplam: {toplam}")
print(f"Toplam: {toplam:.2f} TL")
print(f"Toplam: {toplam:,.2f} TL")
```

Python Ekran Çıktısı

```
Toplam: 749.7
Toplam: 749.70 TL
Toplam: 749.70 TL
```

> **İpucu:** Para, ortalama, ölçüm gibi değerlerde `{deger:.2f}` kalıbı çok kullanılır. `0.1 + 0.2` gibi işlemlerde çıkan `0.30000000000000004` türü uzun sayıları da `:.2f` ile düzgün gösterebiliriz.

20.3.5. f-string ile Hizalama

Metni belirli bir genişlikte sola (<), sağa (>) veya ortaya (^) hizalayabiliriz. Tablo gibi düzenli çıktılar için kullanılır.

f-string Hizalama Kalıpları

Kalıp	Örnek	Sonuç (| işaretleri sınırı gösterir)	Açıklama
`:<10`	`f"{'Elma':<10}"`	`|Elma      |`	10 karakterlik alanda sola yasla.
`:>10`	`f"{'Elma':>10}"`	`|      Elma|`	10 karakterlik alanda sağa yasla.
`:^10`	`f"{'Elma':^10}"`	`|   Elma   |`	10 karakterlik alanda ortala.

Örnek Python Kodu Hizalı fiyat listesi.

```python
print(f"{'Ürün':<10}{'Fiyat':>8}")
print(f"{'Elma':<10}{12.5:>8.2f}")
print(f"{'Armut':<10}{9.75:>8.2f}")
print(f"{'Muz':<10}{30:>8.2f}")
```

Python Ekran Çıktısı

```
Ürün         Fiyat
Elma         12.50
Armut         9.75
Muz          30.00
```

20.3.6. f-string'de Süslü Parantez ve Tırnak Kullanımı

- Metnin içinde süslü parantezin kendisini göstermek istersek **iki kez** yazarız: `{{` ve `}}`.
- `{ }` içine doğrudan bir yazı yazacaksak, f-string'in dışındaki tırnaktan **farklı** tür tırnak kullanırız. Dışta çift tırnak varsa içeride tek tırnak kullanırız.

```python
print(f"Küme: {{1, 2, 3}}")        # Küme: {1, 2, 3}

print(f"Ders: {'Python'}")         # Ders: Python
```

20.3.7. Hata Ayıklarken Kullanışlı: {degisken=}

Süslü parantez içine değişkenin sonuna = koyarsak Python hem değişkenin **adını** hem **değerini** yazar. Kodu test ederken çok işe yarar.

```python
puan = 75
ort = 82.5
print(f"{puan=}")
print(f"{ort=}")
```

Python Ekran Çıktısı

```
puan=75
ort=82.5
```

20.4. print() Fonksiyonunun sep ve end Parametreleri

`print()` fonksiyonunun, biz yazmasak da varsayılan değerlerle çalışan iki isteğe bağlı parametresi vardır. Bunları yazarak çıktının biçimini değiştirebiliriz.

print() Parametreleri

Parametre	Varsayılan değer	Ne işe yarar?
`sep` (separator)	`" "` (bir boşluk)	Virgülle ayrılan değerlerin arasına ne konacağını belirler.
`end`	`"\n"` (alt satıra geç)	Yazdırma bittikten sonra satırın nasıl sonlanacağını belirler.

20.4.1. sep: Değerler Arasına Ne Gelsin?

Örnek Python Kodu Farklı ayırıcılarla yazdırma.

```python
print("Pazartesi", "Salı", "Çarşamba")
print("Pazartesi", "Salı", "Çarşamba", sep="-")
print("Pazartesi", "Salı", "Çarşamba", sep=", ")
print("Pazartesi", "Salı", "Çarşamba", sep="")
print(14, 30, 45, sep=":")
```

Python Ekran Çıktısı

```
Pazartesi Salı Çarşamba
Pazartesi-Salı-Çarşamba
Pazartesi, Salı, Çarşamba
PazartesiSalıÇarşamba
14:30:45
```

20.4.2. end: Satır Nasıl Bitsin?

Normalde her `print()` işini bitirince alt satıra geçer. `end` ile bu davranışı değiştirebiliriz.

Örnek Python Kodu Aynı satırda yazdırmaya devam etme.

```
print("Yükleniyor", end="...")
print("Tamam")

print("A", end=" ")
print("B", end=" ")
print("C")
```

Python Ekran Çıktısı

```
Yükleniyor…Tamam
A B C
```

`sep` ve `end` birlikte de kullanılabilir:

```
print("Gün", "Ay", "Yıl", sep="/", end=" <- tarih biçimi\n")
# Gün/Ay/Yıl <- tarih biçimi
```

20.5. Kaçış Karakterleri (Escape Characters)

Metin içinde yeni satır, sekme (tab) veya tırnak işareti gibi özel karakterler kullanmak için ters eğik çizgi `\` ile başlayan **kaçış karakterleri** kullanılır.

Sık Kullanılan Kaçış Karakterleri

Karakter	Anlamı	Örnek
`\n`	Yeni satıra geç.	`print("A\nB")`
`\t`	Sekme (tab) boşluğu bırak.	`print("A\tB")`
`\\`	Ters eğik çizgi karakterini yazdır.	`print("C:\\Okul")`
`\"`	Çift tırnak karakterini yazdır.	`print("O \"evet\" dedi")`
`\'`	Tek tırnak karakterini yazdır.	`print('Ali\'nin kitabı')`

```
print("Birinci\nİkinci")
print("Ad:\tAyşe")
print("Yol: C:\\Okul")
print("O \"evet\" dedi")
```

Python Ekran Çıktısı

```
Birinci
İkinci
Ad:   Ayşe
Yol: C:\Okul
O "evet" dedi
```

20.6. Uygulama Örnekleri

Örnek 1: Daire alanı hesaplama

```
pi = 3.14159
r = float(input("Yarıçap (cm): "))
alan = pi * r * r
print(f"r={r}cm olan dairenin alanı {alan:.2f}cm²'dir.")
```

Python Ekran Çıktısı

```
Yarıçap (cm): 5
r=5.0 cm olan dairenin
alanı 78.54 cm²'dir.
```

Örnek 2: Not ortalaması

```python
ad = input("Öğrenci adı: ")
vize = float(input("Vize notu: "))
final = float(input("Final notu: "))

ortalama = vize * 0.4 + final * 0.6

print(f"Öğrenci : {ad}")
print(f"Ortalama: {ortalama:.1f}")
print(f"Geçti mi: {ortalama >= 50}")
```

Python Ekran Çıktısı

```
Öğrenci adı: Ayşe
Vize notu: 70
Final notu: 90
Öğrenci : Ayşe
Ortalama: 82.0
Geçti mi: True
```

Örnek 3: Market fişi

```python
urun = input("Ürün adı: ")
fiyat = float(input("Birim fiyat: "))
adet = int(input("Adet: "))

toplam = fiyat * adet

print("-" * 24)
print(f"{urun:<12}{adet:>3} x {fiyat:>6.2f}")
print(f"{'TOPLAM':<12}{toplam:>11.2f} TL")
print("-" * 24)
```

Python Ekran Çıktısı

```
Ürün adı: Defter
Birim fiyat: 24.5
Adet: 4
------------------------
Defter        4 x  24.50
TOPLAM            98.00 TL
------------------------
```

Saniyeyi saat-dakika-saniyeye çevirme

```python
toplam = int(input("Saniye giriniz: "))

saat = toplam // 3600
kalan = toplam % 3600
dakika = kalan // 60
saniye = kalan % 60

print(f"{saat:02d}:{dakika:02d}:{saniye:02d}")
```

Python Ekran Çıktısı

```
Saniye giriniz: 3725
01:02:05
```

Özet:

- Fonksiyon, bir işi yapan isimli kod parçasıdır; `isim(argümanlar)` biçiminde çağrılır ve gerekirse bir dönüş değeri verir.
- `input()` klavyeden veri alır ve **her zaman metin** döndürür. Sayı gerekiyorsa `int()` veya `float()` ile dönüştürülür.
- `print()` değerleri ekrana yazar. Virgül araya boşluk koyar; **f-string** ise değişkenleri metnin içine `{ }` ile yerleştirir.

III Python `type()` Fonksiyonu

✱ Python'da her değerin bir türü vardır; `type()` bu türü bize söyler.

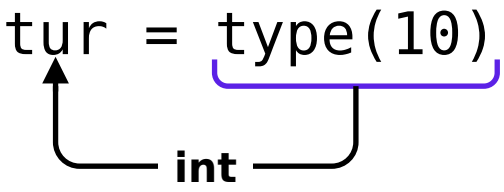
1. `type()` Nedir?

Python'da her değer (sayı, metin, liste, hatta fonksiyonlar) bir **nesnedir** ve her nesne bir **türe (sınıfa)** aittir. `type()` fonksiyonu, kendisine verilen nesnenin türünü döndürür.

Tanım

`type(nesne)` ifadesi, nesnenin ait olduğu sınıfı döndürür. Dönen değer bir değişkene atanabilir veya doğrudan yazdırılabilir.

Aşağıdaki şemada `type(10)` çağrısının sonucu `tur` değişkenine nasıl aktarılıyor, görebilirsin. Fonksiyon `int` sınıfını döndürür.



Örnek 1: Temel kullanım

```
1  print(type(10))
2  print(type(3.14))
3  print(type("Merhaba"))
4  print(type(True))
```

Ekran Çıktısı

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Çıktıdaki `<class 'int'>` ifadesi şu anlama gelir: "Bu değer, `int` sınıfından bir nesnedir."

2. Temel Veri Türleri

Sık Karşılaşılan Türler

Değer	type() sonucu	Açıklama
`42`	`int`	Tam sayı
`3.14`	`float`	Ondalıklı sayı
`"Python"`	`str`	Metin (string)
`True`	`bool`	Mantıksal değer
`[1, 2, 3]`	`list`	Liste
`(1, 2, 3)`	`tuple`	Demet
`{"a": 1}`	`dict`	Sözlük
`{1, 2, 3}`	`set`	Küme
`None`	`NoneType`	Değer yokluğu

3. Neden İhtiyaç Duyarız?

Python **dinamik tipli** bir dildir. Bir değişkenin türünü önceden yazmayız; Python değere bakarak türü kendisi belirler. Üstelik aynı değişken sonradan başka türde bir değer alabilir.

Örnek 2: Değişkenin türü değişebilir

```
1  x = 5
2  print(type(x))
3
4  x = "beş"
5  print(type(x))
```

Ekran Çıktısı

```
<class 'int'>
<class 'str'>
```

Tür önceden yazılmadığı için, bir değişkenin o anda hangi türde olduğunu görmek istediğimizde `type()` kullanırız. Hata ayıklarken (debug) en sık başvurulan araçlardan biridir.

3.1. Tipik hata: `input()` her zaman `str` döndürür

```
1  yas = input("Yaşınız: ")
2  print(type(yas))
3
4  yas = int(yas)
5  print(type(yas))
```

Ekran Çıktısı (kullanıcı 15 girdiğinde)

```
Yaşınız: 15
<class 'str'>
<class 'int'>
```

Dikkat: Kullanıcı `15` yazsa bile `input()` bunu `"15"` yani **metin** olarak verir. Matematiksel işlem yapmadan önce `int()` veya `float()` ile dönüştürmek gerekir.

4. Tür Kontrolü: `type()` ve `isinstance()`

Bir değerin belirli bir türde olup olmadığını kontrol etmek için iki yol vardır.

```python
sayi = 10

# Yöntem 1: type() ile karşılaştırma
if type(sayi) == int:
    print("Bu bir tam sayı")

# Yöntem 2: isinstance() ile kontrol
if isinstance(sayi, int):
    print("Bu bir tam sayı")
```

İkisi de aynı sonucu verir, ancak aralarında önemli bir fark vardır: `isinstance()` **miras (inheritance)** ilişkisini de dikkate alır, `type() ==` almaz.

```python
print(type(True) == int)
print(isinstance(True, int))
```

Ekran Çıktısı

```
False
True
```

Python'da `bool` sınıfı, `int` sınıfından türetilmiştir. Yani `True` aslında özel bir tam sayıdır. `isinstance()` bunu doğru şekilde `True` olarak döndürür.

✱ Tür kontrolü için `isinstance()`, türü görmek için `type()` kullan.

4.1. Birden fazla tür kontrol etmek

```python
deger = 3.5
if isinstance(deger, (int, float)):
    print("Bu bir sayı")
```

5. Türü Kullanarak Nesne Oluşturmak

`type()` ile elde ettiğimiz tür, aslında çağrılabilir bir sınıftır. Dolayısıyla türün kendisini kullanarak yeni bir nesne üretebiliriz.

```python
a = 5
b = type(a)("12")    # type(a) -> int, yani int("12")
print(b)
print(type(b))
```

Ekran Çıktısı

```
12
<class 'int'>
```

6. İleri Düzey: Üç Argümanlı `type()`

`type()` aslında iki farklı biçimde çalışır.

Kullanım	Ne yapar?
`type(nesne)`	Nesnenin türünü döndürür.
`type(ad, tabanlar, ozellikler)`	Dinamik olarak yeni bir **sınıf** oluşturur.

Aşağıdaki iki tanım birbirine eşdeğerdir.

```python
# Klasik yol
class Ogrenci:
    okul = "MAETMTAL"

# type() ile aynı sınıf
Ogrenci = type("Ogrenci", (), {"okul": "MAETMTAL"})

o = Ogrenci()
print(o.okul)
print(type(o))
```

Ekran Çıktısı

```
MAETMTAL
<class '__main__.Ogrenci'>
```

Şimdilik bilmen gereken şey, Python'da **sınıfların da birer nesne olduğu** ve onların türünün `type` olduğudur.

```python
print(type(int))
print(type(type))
```

Ekran Çıktısı

```
<class 'type'>
<class 'type'>
```

Özet

- `type(nesne)` nesnenin türünü döndürür.
- Python dinamik tipli olduğu için değişkenin türü sonradan değişebilir.
- `input()` her zaman `str` döndürür; dönüşüm yapmayı unutma.
- Tür kontrolü için `isinstance()` daha güvenlidir.
- `type()` üç argümanla çağrılırsa yeni sınıf üretir (ileri düzey).

7. Alıştırmalar

1. Aşağıdaki değerlerin türlerini tahmin et, sonra `type()` ile kontrol et: `7`, `7.0`, `"7"`, `[7]`, `(7,)`, `(7)`.
2. Kullanıcıdan iki sayı alıp toplayan bir program yaz. Önce dönüşüm yapmadan çalıştır, sonucu açıkla; ardından `type()` ile türleri yazdırarak hatayı düzelt.
3. `isinstance(True, int)` ifadesi neden `True` döndürür? Kendi cümlelerinle açıkla.
4. `type(5) == type(5.0)` ifadesinin sonucu nedir? Neden?

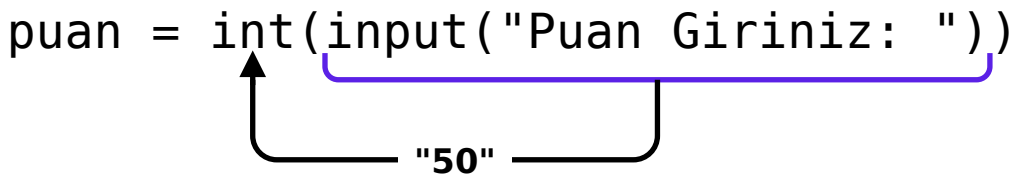
8. `int(input())` Nasıl Çalışır?

```
1 puan = int(input("Puan Giriniz: "))
```

Python bu satırı **içten dışa** doğru çalıştırır: önce en içteki `input()`, sonra onu saran `int()`, en son da atama (=).

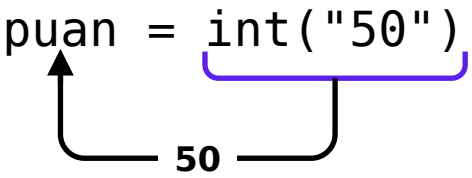
8.1. Adım 1: `input()` çalışır ve `int()` fonksiyonuna gider

Kullanıcı klavyeden `50` yazıp Enter'a basar. `input()` fonksiyonu bu değeri her zaman **metin** olarak döndürür; yani elimize geçen şey sayı değil, tırnak içinde `"50"` olur. Bu değer, dıştaki `int()` fonksiyonuna gider.



8.2. Adım 2: `int()` çalışır ve sonuç `puan` değişkenine gider

`input()` çağrısı yerini döndürdüğü değere bıraktı; satır artık şu hale geldi: `puan = int("50")`. `int()` fonksiyonu `"50"` metnini tam sayıya çevirir ve sonuç olan `50` değeri `puan` değişkenine atanır.



8.3. Sonuç

```
1 puan = int(input("Puan Giriniz: "))
2 print(puan)
3 print(type(puan))
```

Ekran Çıktısı

```
Puan Giriniz: 50
50
<class 'int'>
```

Dikkat: `int()` olmasaydı `puan` değişkeninde `"50"` yani **metin** olurdu ve `puan + 10` gibi bir işlem hata verirdi. Sayıyla işlem yapacaksak `int()` (ya da ondalıklı değerler için `float()`) ile dönüştürmemiz gerekir.